# Modified-Inverse Physics Informed Neural Networks for Determination of Orthotropic Thermal Conductivities

G. Sai Krishna[1], Jithu J[1], Muthu Saran[1], C Balaji[1], and Balaji Srinivasan[1,2]

[1]Heat Transfer and Thermal Power Laboratory, Department of Mechanical Engineering, Indian Institute of Technology Madras, Chennai, India-600036
[2]Wadhwani School of Artificial Intelligence, Indian Institute of Technology Madras, Chennai, India-600036

**Abstract**

**Purpose**
This study proposes a modified inverse physics-informed neural network (MIPINN) framework for accurate and efficient simultaneous multiparameter identification in three-dimensional heat conduction problems under sparse experimental data conditions.
**Design/methodology/approach**
The proposed MIPINN framework enhances conventional inverse-PINN formulations by integrating finite difference-based pseudo gradient descent optimization within the physics-informed architecture. The methodology is systematically validated through five configurations: (i) two-dimensional numerical simulation considering temperature as the exact solution for strict comparison and methodological assessment, (ii) three-dimensional synthetic temperature field reconstruction with prescribed orthotropic thermal conductivity parameters, (iii) isotropic thermal conductivity estimation of AISI 304 stainless steel using experimentally acquired temperature data, (iv) orthotropic thermal conductivity tensor identification of a lithium-ion pouch battery (AMP20M1HD-A) through controlled thermal characterization experiments and (v) extension of the methodology for determination of specific heat capacity of a three-dimensional slab using synthetically generated data.
**Findings**
Across all investigated cases, the proposed technique demonstrates improved parameter estimation accuracy and computational efficiency compared to conventional inverse-PINN approach. The MIPINN methodology exhibits enhanced robustness, reduced parameter uncertainty, and stable convergence behavior under sparse data constraints even in three-dimensional multiparameter inverse heat conduction scenarios.
**Originality**
This study presents a modified inverse-PINN strategy that strengthens constraint enforcement, thereby improving solution stability and convergence under limited observational data for multiparameter inverse heat conduction problems and demonstrates its effectiveness through comprehensive three-dimensional experimental validation.

**Keywords:** Physics-informed neural networks, inverse methods, orthotropic thermal conductivity, deep learning, pouch-type Li-ion battery

# Nomenclature

| | |
|---|---|
| $b$ | Bias |
| $h$ | Heat transfer coefficient, $W/m^2K$ |
| $k_{ii}$ | Orthotropic thermal conductivity, $W/mK$ |
| $\mathscr{L}$ | Loss function |
| $M$ | Number of sampling points |
| $\mathscr{N}$ | Neuron output |
| $\mathscr{N}\mathscr{N}$ | Neural network function |
| $\mathbf{W}$ | Neural network weight matrix |
| $x$, $y$, $z$ | 3D cartesian coordinates |
| $T$ | Temperature, $K$ |
| $\hat{T}$ | Neural network output temperature |
| $X$ | $x$, $y$, $z$ |

# Greek letters

| | |
|---|---|
| $\alpha$ | Learning rate for training the network weights |
| $\beta$ | Learning rate for training thermal conductivity |
| $\varepsilon$ | Tolerance for convergence of losses |
| $\gamma$ | Stabilizing constant |
| $\lambda$ | Loss weight |
| $\Omega$ | Domain |
| $d\Omega$ | Boundary |
| $\sigma$ | Activation function |

# Subscripts/Superscript

| | |
|---|---|
| $i$ | Epoch index |
| $in$ | Input layer in the neural network |
| $j$ | Sample index |
| $\eta$ | Neural network output layer |
| $m$ | Number of neurons in a layer of neural network |
| $n$ | Neural network layer index |
| $out$ | Output layer in the neural network |
| $xx$ | x-direction |
| $yy$ | y-direction |
| $zz$ | z-direction |

# Abbreviation

| | |
|---|---|
| BC | Boundary conditions |
| IHCP | Inverse heat conduction problem |
| MIPINN | Modified inverse physics informed neural network |
| NN | Neural network |
| PDE | Partial differential equation |
| PINN | Physics informed neural network |

# 1 Introduction

Inverse problems involve determining the underlying characteristics or parameters of a system from the observed consequences it produces. Unlike forward problems, where the system behavior is predicted from known parameters, inverse problems attempt to infer the causes from the effects. These problems are often ill-posed, as the available observations may be insufficient to uniquely and stably determine the governing characteristics that predominantly influence the system's behavior.

In the field of thermal science, thermal management systems (TMS) have emerged as a significant area of research, driven by the increasing use of automation in every field and the resultant surge in machine usage globally, spanning industries from advanced manufacturing to electronics, automotive, aerospace, and energy systems. As the complexity and miniaturization of devices advance, managing thermal loads effectively becomes critical not just for performance, but for safety, reliability, and longevity of components.

The ongoing demand for enhanced heat transfer has led to the development of novel materials, devices, and systems. Progress is evident not only in thermal engineering but across various engineering and technological disciplines. Improved heat transfer capabilities are now critical as temperature plays an essential role in device reliability and failure mitigation. Within the framework of thermal management system, accurately determining the thermal properties of materials and devices is often the first and indeed an essential step for advancing further research. Inverse heat transfer methods have emerged as a solution for not only to characterize the thermal properties of the materials but also to estimate the thermal boundaries of a working system. In the past, various experimental, statistical, and numerical techniques have been developed for determining the required thermal parameters and boundaries such as thermal conductivity, specific heat, heat generation, heat flux, etc. Due to the revolutionary growth in the performance of CPUs and GPUs, machine learning (ML) techniques have gained strength in solving problems in every existing field when provided with a sufficient amount of data. These techniques need an enormous amount of data, which is acquired by running a large number of simulations, consuming a lot of computational effort and space, as there is no physics incorporated into the technique.

Physics-informed neural network (PINN) is an advanced version of artificial neural network (ANN) where the physical interpretation of the problem can be seamlessly incorporated into the network, eliminating dependency on large data and also satisfying the need to incorporate physics into a data-driven machine learning associated problem solving. Leveraging the feature called auto differentiation provided by the major ML frameworks such as PyTorch and Tensorflow, PINNs

have achieved the capability of solving partial differential equations (PDEs), which invariably describe the physical phenomena existing in the field of heat transfer, fluid mechanics, solid mechanics, etc. PINNs are emerging as a promising alternative to numerical methods such as finite difference methods (FDM), finite volume methods (FVM), etc, for solving the forward problems related to computational heat transfer, fluid mechanics, and solid mechanics, as well. Other than forward modelling, researchers have also shown that PINNs have the unique capability of solving the inverse problems related to fluid mechanics and heat transfer.

In this work, a modified and well-tuned finite difference gradient descent-based modified inverse PINN (MIPINN) methodology has been proposed to solve inverse heat transfer problems of considerable engineering interest. The proposed methodology has the potential to simultaneously solve both the forward and inverse problems, which cannot be achieved using well-established traditional inverse algorithms. Brief literature regarding the work has been presented in the Section (2). A detailed explanation for the modified inverse PINNs methodology has been provided in the Section (3). The experimental methodology has been elucidated in Section 4. The proposed methodology was tested on a variety of numerical and experimental cases, and it was found to be highly effective in terms of computational cost and accuracy, outperforming standard inverse PINN technique. Results of the test cases are presented in Section (5).

# 2 Brief review of inverse techniques and PINNs

Early inverse heat conduction problems (IHCP) focused on estimating heat influx history for missile re-entry vehicles using internal temperature measurements [1]. Sawaf and Ozisik [2] employed conjugate gradient and Levenberg-Marquardt methods for orthotropic thermal conductivity, while Monde [3] used Laplace transforms for surface temperature and heat flux estimation. Liu [4] introduced a group preserving scheme for temperature-dependent thermophysical quantities. Mierzwiczak and Kołodziej [5] applied the golden search method for 2D steady IHCPs, and Czel, Grof [6], and Liu [7] used genetic algorithms for temperature-dependent conductivity and heat source estimation. Hematiyan et al. [8] utilised boundary element analysis for anisotropic conductivities. Bayesian approaches were also employed, as in Wang and Zabaras [9] with hierarchical models for thermal parameters and boundary fluxes.

Zhu et al. [10] leveraged deep learning for boundary condition estimation in transient IHCPs, while Suraj and Balaji used Bayesian MH-MCMC for SS304 materials. Suraj et al. [11] extended this to Li-ion batteries. Hafid et al. [12] retrieved tumour tissue properties using Levenberg-Marquardt and Broyden methods, and Chanda et al. [13] used ANN-based inverse methods for composites. Huang and Jan [14] adopted conjugate gradient for time-dependent properties, and Mazian et al. utilised BFGS for thermal parameters.

PINNs, introduced by Raissi et al. [15], demonstrated their ability to solve both forward and inverse problems by tackling a wide range of partial differential equations (PDEs) governing various physical phenomena. Variants such as Distributed PINNs [16], Extended PINNs [17], and Transfer PINNs [18] address stationary, transient, and heterogeneous problems. PINNs have been applied in shock-capturing [19, 20], fluid mechanics [21], two-phase flows [22], and industrial heat transfer. Zhang et al. [23] have developed a parallel self-adaptive loss weighting algorithm for achieving balanced convergence among the PINN losses, while Wang et al. [24] demonstrated adaptive deep PINNs that can avoid the loss function converging at a local minima Applications extend to porous

media [25], phase change [26], and convection-dominant problems [27]. Ma et al. used adaptive fractional PINNs for transient heat conduction. Jha and Mallik [28] applied Fourier-feature PINNs for nonlinear diffusion. Zeng et al. [29] and Rui et al. [30] reconstructed flow fields in engineering scenarios. PINNs are also used in solid mechanics [31, 32], reaction-diffusion systems, biomedical engineering, and renewable energy [33, 34]. Li et al. [35] developed a ConvLSTM-PINN hybrid network with adaptive loss weighting specifically for space-time fractional tsunami modeling. Oommen and Srinivasan [36] found PINNs computationally superior for 1D inverse heat transfer. Billah et al. [37] and He et al. [38] estimated thermal flux and parameters in unsteady heat conduction and multi-parameter cases. Papadopoulos et al. [39] used XPINNs for determining interface thermal resistance in composites. Jo et al. [40] used the inverse PINN for estimating the thermal conductivities of both numerical and experimental cases, they have also predicted the orthotropic components of thermal conductivity of 18650 Li-ion battery cell using their methodology. Liu et al. [41] used PINNs for predicting the boundary conditions of a rocket motor using the experimental data. Beardi et al. [42] estimated the parameters of one-dimensional transient diffusion and advection-diffusion equation, wherein they employed an adaptive loss weightage scheme for expediting the PINN algorithm.

Although the conventional inverse PINN approach has been applied in multiple cases, it appears to be more susceptible to ill-posedness when addressing simultaneous multi-parameter estimation problems, thereby reducing the likelihood of convergence to accurate solutions. Furthermore, to the best of the authors' knowledge, a very few studies [40] have demonstrated real-time single-parametric estimation using experimental validation.

This work introduces a novel, efficient, and robust modified inverse-PINN (MIPINN) methodology with stronger constraints, which greatly limits solution flexibility while improving convergence even in three-dimensional multi-parameter inverse problems. Additionally, the methodology was validated by estimating the thermal conductivity of an SS304 slab and further applied to determine the orthotropic components of thermal conductivity of a pouch-type Li-ion battery using temperature data from in-house experiments.

# 3 Methodology

The steady-state orthotropic heat conduction process is governed by the equation (1). Here, a modified inverse-PINN framework is designed for the estimation of orthotropic thermal conductivity components $k_{xx},\ k_{yy},\ and\ k_{zz}$ where $T$ represents the temperature and $q'''$ represents volumetric heat generation which are functions of the spatial coordinates $X = (x,\ y,\ z)$.

$$k_{xx}\frac{\partial^2 T(X)}{\partial x^2} + k_{yy}\frac{\partial^2 T(X)}{\partial y^2} + k_{zz}\frac{\partial^2 T(X)}{\partial z^2} = q'''(X); \quad X \in (x,y,z) \in \mathbb{R}^3 \tag{1}$$

## 3.1 Physics Informed Neural Networks (PINNs)

Artificial neural networks (ANNs) are powerful supervised computational models designed to capture complex, nonlinear relationships within data. Built from layers of interconnected processing units (neurons), ANNs transform input data through successive computations to generate meaningful outputs. Due to their adaptability and ability to generalise, these models demonstrate ex-

ceptional effectiveness across various applications, such as regression, pattern recognition, and complex function approximation. The foundational work by Hornik et al. [43] established the universal approximation property of ANNs, demonstrating their theoretical potential to approximate any continuous function. This theoretical grounding, combined with advances in training algorithms and computational resources, has enabled ANNs to achieve remarkable success in applications such as image recognition, natural language processing, speech analysis, etc. In the field of computational fluid dynamics (CFD), ANNs are widely used to improve simulation performance or to build reduced-order models for complex cases. Simulation or experimental results serve as the learning resource for building robust reduced-order NN models for CFD.

The architecture of an ANN consists of an input layer, one or more hidden layers, and an output layer. The ANN receives $x_1,\ x_2,\ \ldots\ x_{m_{in}}$ which are further passed through hidden layers, where neurons apply weighted computations and nonlinear activation functions. The processed information flows forward through the network, ultimately producing the output $\hat{T}$. Each layer contributes to learning the complex relationship between the inputs and the desired output.

The neural network function $\mathscr{NN}$ is regarded as an approximate solution $\hat{T}$ to the true solution $T$. As demonstrated by equation (2)[44], the neural network operates on input $X = [x_1, x_2, \ldots, x_{m_{in}}] \in \mathbb{R}^{m_{in}}$. Output of $\eta$ layered neural network comprising $m_n$ neurons in the $n$-th layer is defined as $\mathscr{N}^{\eta}(X) : \mathbb{R}^{m_{in}} \to \mathbb{R}^{m_{out}}$. The weight matrix and bias vector for the $n$-th layer are denoted by $\mathbf{W}^n \in \mathbb{R}^{m_n \times m_{n-1}}$ and $\mathbf{b}^n \in \mathbb{R}^{m_n}$, respectively.

$$\hat{T} = \mathscr{NN}^{\eta}(X) = \text{Layers of } [\mathscr{N}^0(X) \quad \mathscr{N}^n(X) \quad \mathscr{N}^{\eta}(X)] \tag{2}$$

The output of every layer $\mathscr{N}$ in the forward pass is recursively defined as follows:

- Input layer: $\mathscr{N}^0(X) = X \in \mathbb{R}^{m_{in}}$,
- Hidden layer : $\mathscr{N}^n(X) = \sigma(\mathbf{W}^n \mathscr{N}^{n-1}(X) + \mathbf{b}^n) \in \mathbb{R}^{m_n} \quad \text{for} \quad 1 \leq n \leq \eta - 1$,
- Output layer: $\mathscr{N}^{\eta}(X) = \mathbf{W}^{\eta} \mathscr{N}^{\eta-1}(X) + \mathbf{b}^{\eta} \in \mathbb{R}^{m_{out}}$.

Here, $\sigma$ denotes the activation function, which introduces non-linearity into the neural network (NN). The activation function is applied elementwise to the output of each neuron, enabling the network to learn complex mappings. The performance of the NN is evaluated using a loss function $\mathscr{L}$, typically the mean squared error (MSE), defined as $\frac{1}{M}\sum_0^M (T_j - \hat{T}_j)^2$ over $M$ sampling points, where $T_j$ represents the true values and $\hat{T}_j$ the predicted outputs. The goal is to minimize this loss, indicating a better fit of the network to the data. To achieve this, the weights of the network, $\mathbf{W}^n$, are iteratively updated so that the loss approaches a minimal value. This process is referred to as back propagation or learning. Optimization is generally carried out using the gradient descent algorithm (3) and its advanced versions. which updates the weights in the direction of the negative gradient of the loss function. The process of updating the network weights $\mathbf{W}^n$ using the simple gradient descent algorithm is mathematically expressed as

$$\mathbf{W}^{n,i} = \mathbf{W}^{n,i-1} - \alpha \nabla_{\mathbf{W}} \mathscr{L}, \tag{3}$$

where $\nabla_{\mathbf{W}}$ is gradient operator with respect to the weights and $i$ is the epoch counter. The form for any variant of the gradient descent algorithm can be expressed as the following equation (4)

$$\mathbf{W}^{n,i} = \mathbf{W}^{n,i-1} - \delta \mathbf{W}^{n,i}, \tag{4}$$

Physics-informed neural networks (PINNs) represent an enhanced form of ANN that embeds physical laws, typically expressed as partial differential equations (PDEs), into the learning framework. Unlike conventional ANNs, the distinguishing feature of PINNs lies in their formulation of the loss function ($\mathscr{L}$), which incorporates the residuals of both the governing PDEs and the boundary conditions (BCs). This nudges the network to produce solutions that are consistent with the underlying physics. The PDE guides the NN to get the right trend for the solution, and the BC helps the solution to fit in the right boundary.

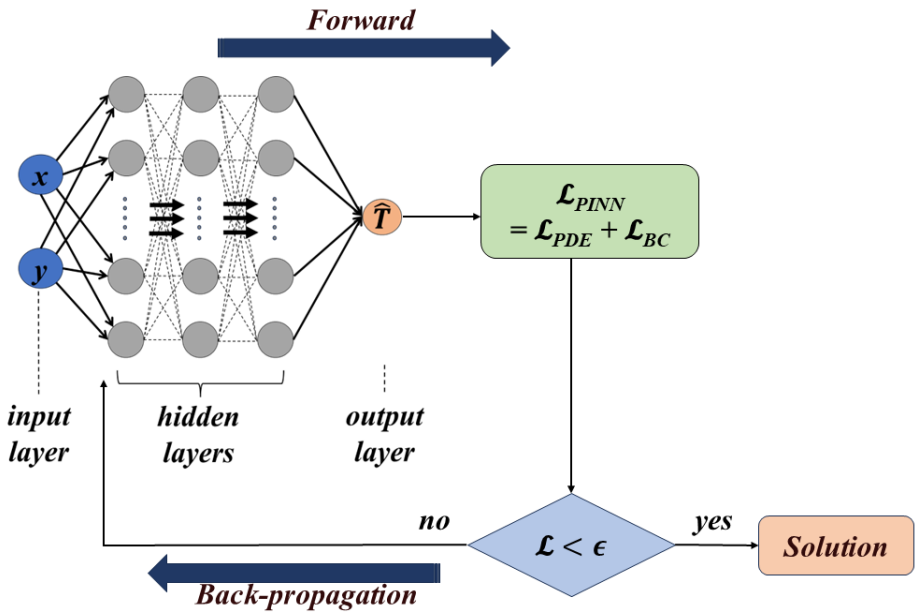



Figure 1: Graphical representation of a PINN architecture showing forward and backward-propagation

The input to a PINN generally consists of spatial coordinates $(x,\ y,\ z)$ and time $t$, while the output corresponds to the solution of the governing PDE. A central aspect of PINNs is their ability to compute differential terms present in the PDEs and BCs using automatic differentiation, a technique supported by major deep learning libraries such as TensorFlow [45] and PyTorch [46]. The total loss function ($\mathscr{L}$) is constructed from the residuals of the PDE and the available boundary or initial condition data. The network is then trained using back-propagation to minimize this loss. This physics-based learning approach allows PINNs to model complex physical systems without relying on traditional numerical solvers or mesh-based discretization methods. In essence, PINNs merge data-driven learning with physical laws, providing a powerful tool for solving a wide range of problems in science and engineering, from fluid dynamics and heat transfer to solid mechanics and beyond. Figure 1 depicts the schematic of the PINN architecture. Based on the governing equation (1) in this study, spatial coordinates $(x,y,z)$ are given as input to the PINN architecture and predict $\hat{T}$ as the output.

The main advantages of PINN are: 1) Suitable for multi-dimensional simulations, including transient problems. 2) More stable than explicit numerical methods. 3) Requires less storage since only network architecture is stored, unlike CFD which needs mesh and solution data. 4) Effective for building global surrogate models with limited data. 5) Easier code implementation

for complex, multi-dimensional simulations. 6) Well-suited to inverse problems with sparse data. 7) Avoids discretization and truncation errors. 8) Preserves higher-order space–time solutions without the method of lines approximation. 9) Provides continuous output functions, unlike point-based numerical solutions. 10) No need for mesh connectivity information. 11) Less prone to overfitting compared to ANN-based surrogate models. [19]

While PINN has multiple benefits, it also has some limitations. 1) Higher computational costs than conventional numerical methods. 2) Optimal configuration for specific problems may require significant tuning. 3) Optimization algorithms might converge to local minima. 4) May result in multiple solutions if boundary or initial conditions are not selected appropriately. 5) Validation is needed across various complex problems. 6) Convergence issues may arise from improper parameter selection or network design. 7) The use of costly GPU resources may be necessary depending on the problem.

### 3.1.1 Loss definitions

In the PINN framework, the loss term $\mathscr{L}$ serves as a comprehensive metric that combines all individual loss components, such as PDE loss, boundary losses, and data loss, into a single objective. By summing these distinct residuals, the total loss ensures that the neural network solution simultaneously satisfies the underlying physics, adheres to boundary conditions, and aligns with observed data. Typically, the mean absolute error (MAE) is used as the metric to quantify all the losses in this work, providing a straightforward measure of the average magnitude of prediction errors across all components. Following are the different losses and their definitions in the context of the problems considered in this study.

- **PDE Loss**

  Equation (1) is modified upon substituting the NN output variable $\hat{T}$ and can be expressed as shown in the equation (5)

$$k_{xx}\frac{\partial^2\hat{T}(X)}{\partial x^2}+k_{yy}\frac{\partial^2\hat{T}(X)}{\partial y^2}+k_{zz}\frac{\partial^2\hat{T}(X)}{\partial z^2}=q'''(X),\quad X\in\Omega\in\mathbb{R}^3 \tag{5}$$

  As $\hat{T}$ does not precisely satisfy equation (5) at the given location $X$ for the initial network weights, this results in a non-zero residual. By integrating the residuals across all the $M_\Omega$ collocation points within the domain $\Omega$ and computing their mean absolute value, the resulting PDE loss $\mathscr{L}_{PDE}$ is obtained, as expressed in equation (6):

$$\mathscr{L}_{PDE}=\frac{1}{M_\Omega}\sum_{j=1}^{M_\Omega}\left|\left[k_{xx}\frac{\partial^2\hat{T}(X_j)}{\partial x^2}+k_{yy}\frac{\partial^2\hat{T}(X_j)}{\partial y^2}+k_{zz}\frac{\partial^2\hat{T}(X_j)}{\partial z^2}-q'''(X)\right]\right|,\quad X\in\Omega\in\mathbb{R}^3 \tag{6}$$

- **Boundary Loss**

  Boundary loss or residual $\mathscr{L}_{BC}$ can also be defined depending on the kind of BC, Dirichlet, Neumann, or mixed type of boundary.

– **Dirichlet boundary loss** on the designated boundary $d\Omega$, where solution variable $T$ is prescribed as $f(X)$, is expressed as shown in the equation (7):

$$\mathscr{L}_{BC} = \frac{1}{M_{d\Omega}} \sum_{j=1}^{M_{d\Omega}} \left| \left[ \hat{T}(X_j) - f(X_j) \right] \right|, \quad X \in d\Omega \in \mathbb{R}^3 \tag{7}$$

This form of boundary condition applies when the temperature along the boundary is specified as a function of spatial location.

– **Neumann boundary loss** on the given boundary $d\Omega$, where the normal gradient of the solution variable $\frac{\partial \hat{T}}{\partial X_j}$is prescribed as $f(X)$ can be expressed as equation (8):

$$\mathscr{L}_{BC} = \frac{1}{M_{d\Omega}} \sum_{j=1}^{M_{d\Omega}} \left| \left[ \frac{\partial \hat{T}}{\partial X_j} - f(X_j) \right] \right|, \quad X \in d\Omega \in \mathbb{R}^3 \tag{8}$$

This type of boundary condition is employed when a specific heat flux is applied at the boundary.

– **Mixed boundary loss** is generated when the linear combination of Dirichlet and Neumann conditions are on the given boundary $d\Omega$ and can be defined as given in the equation (9):

$$\mathscr{L}_{BC} = \frac{1}{M_{d\Omega}} \sum_{j=1}^{M_{d\Omega}} \left| \left[ \frac{\partial \hat{T}(X_j)}{\partial X_j} + m\hat{T}(X_j) - f(X_j) \right] \right|, \quad X \in d\Omega \in \mathbb{R}^3 \tag{9}$$

where m is a constant. This type of boundary condition typically applies to domain boundaries exposed to ambient conditions and is referred to as a convective-conductive boundary.

- **Data loss**

Data loss $\mathscr{L}_{Data}$ quantifies the discrepancy between model predictions and observed measurement data at specific sensor locations. It serves as an essential component for guiding the inverse estimation process within the PINN framework. The data loss residual at measurement points $X^{(data)}$ is defined as shown in the equation (10)

$$\mathscr{L}_{Data} = \frac{1}{M_{Data}} \sum_{j=1}^{M_{Data}} \left| \left[ \hat{T}(X_j^{(Data)}) - T(X_j^{(Data)}) \right] \right|, \tag{10}$$

where $\hat{T}(X^{(Data)})$ is the temperature predicted by the neural network at $M_{Data}$ sensor locations, and $T(X^{(Data)})$ is the corresponding observed value from experiments or measurements. Minimizing this loss enforces agreement between the model and empirical data, thereby enhancing the reliability for obtaining the solution.

Typically total loss for PINNs, $\mathscr{L}_{PINN}$ is defined as the sum of PDE and the boundary losses (without the data loss) as shown in the following equation (11):

$$\mathscr{L}_{PINN} = \lambda_P \cdot \mathscr{L}_{PDE} + \lambda_B \cdot \mathscr{L}_{BC} \tag{11}$$

where $\lambda_P$ and $\lambda_B$ represent the weights given to the PDE loss and the boundary loss, respectively. These weights are assigned to maintain a balance among the simultaneous convergence of PDE and boundary losses. Their values are established through empirical testing and assessment of convergence behaviour.

This $\mathscr{L}_{PINN}$ will then be used as the loss function for optimizing the network weights using the expression shown in equation (3) and run iteratively till all the losses reach an acceptable converged value. This whole process is referred to as forward modelling. The data loss component $\mathscr{L}_{Data}$ will be utilised for the inverse modelling, which will be discussed in the next section.

## 3.2 Modified Inverse-Physics Informed Neural Networks

In addition to forward modelling, PINNs possess the unique capability to identify parameters or coefficients of the differential terms present in PDEs. Generally, inverse problems are inherently ill-posed due to insufficient constraints. To address this, we consider not only PDE loss and boundary losses but also incorporate known data or measurements for the loss function $\mathscr{L}$ as shown in the equation (12) to constrain the solution corresponding to the true parameters. Practically, the volume of this data is limited, acquired through real-time experiments. Consequently, all cases examined in this work utilized a minimal amount of data for estimating the quantity of interest, which is thermal conductivity in this case.

$$\mathscr{L} = \lambda_P \cdot \mathscr{L}_{PDE} + \lambda_B \cdot \mathscr{L}_{BC} + \lambda_D \cdot \mathscr{L}_{Data} \tag{12}$$

The conventional inverse problem workflow with PINNs involves defining the domain geometry, selecting collocation and boundary points, and gathering measurement data for defining the loss function $\mathscr{L}$. The neural network is then configured with spatial coordinates as inputs and the PDE solution as output.

Based on prior knowledge of the domain, the PDE parameters $k_{ii}$ are initialized with educated guess values. For the given problem, the PDE loss, boundary loss, and data loss are evaluated, and the neural network weights $\mathbf{W}$ are updated by minimizing the total loss composed of these three components. In conventional inverse-PINN frameworks, the unknown PDE parameters $k_{ii}$ are treated as trainable neural network parameters and are optimized simultaneously with the network weights. Consequently, both the solution field and the unknown parameters are learned concurrently by minimizing the total loss until the total loss reaches a prescribed tolerance $\varepsilon$.

In most PINN formulations, the weight assigned to the data loss term, $\lambda_D$, is typically chosen to be very small in order to prevent the PDE and boundary residuals from converging to undesirably large values. While this strategy is generally effective for forward problems, where enforcing the governing physics is the primary objective, but introduces significant challenges in inverse settings.

For inverse problems, accurate recovery of the unknown parameters requires simultaneous minimization of both the physics-based PINN loss and the data loss. However, when $\lambda_D$ is very small, the total loss may converge to a low value even though the data loss has not sufficiently decreased. This imbalance can cause the optimization to prioritize reducing the PDE residual while neglecting the data fit, ultimately leading the inferred parameters to converge to incorrect or physically irrelevant solutions. Consequently, the obtained solution becomes ambiguous and unreliable for successful parameter identification.

In the context of multi-parameter inverse problems, particularly when the differences among the parameters are large, the difficulty associated with parameter identification becomes significantly greater. During the training phase, the neural network weights often fail to accurately capture the underlying physical laws governing the system. This is further complicated by the inherent ill-posed nature of inverse problems, where insufficient constraints make the solution space ambiguous. Since the convergence of the unknown parameters is driven by automatic differentiation, where the parameter gradients are explicit functions of the evolving network weights, the process becomes highly sensitive to inaccuracies in those weights during the early stages of training. This causes the inferred parameters to propagate in a largely uncontrolled or random manner, increasing the likelihood that the optimization will deviate from the true values and settle into unstable or physically irrelevant solutions. This trend will become evident from the observations presented in Case 1 of Section 5. Moreover, the complexity of the inverse problem increases further with the dimensionality of the system. In some studies [38], researchers have considered $k_{ii}$ as an output along with the NN output variable $\hat{T}$. This approach doubles the number of network weights at the final layer, increasing computational cost.

The Modified Inverse Physics-Informed Neural Networks (MIPINNs) framework enhances both efficiency and robustness by introducing a dedicated optimization loop for unknown PDE parameters. This loop is governed by the data-loss component, ensuring that the parameter updates are closely tied to the observed behaviour of the data loss. The motivation for this modification arises from the need to effectively monitor the propagation of PDE parameters with data loss, drawing inspiration from momentum-based history-tracking optimization techniques and first-order backward finite difference methods. So the convergence of the inverse parameters is handled through a history-based pseudo-gradient mechanism.

In this formulation, the updates to the network weights follow the standard gradient-descent procedure, whereas the convergence of physics parameters are decoupled and made sensitive only to the magnitude and temporal behaviour of the data loss $\mathscr{L}_{Data}$ with respect to parameter $k_{ii}$, as shown in equation (13). The parameter-update rule is designed in a gradient-descent like form, but instead of relying on automatic differentiation, it employs a pseudo-gradient term, denoted by $\delta k_{ii}$ constructed from the historical evolution of the data loss using the backward finite difference technique. This estimates the gradient of the data loss with respect to the unknown parameters by examining changes across consecutive iterations as described in Equation (14). At each step, this pseudo-gradient is calculated as the ratio between the change in data loss and the change in parameter values from the previous to the current iteration. This method allows for the estimation of a descent direction based on the observed optimization trajectory, without relying on automatic differentiation.

The pseudo-gradient is so named because it depends on the historical sequence of parameter and data-loss values, rather than on the current network parameters. This approach is adopted because $k_{ii}$ does not appear in the computational graph of the data-loss term. Specifically, $\mathscr{L}_{Data} = f(\hat{T}, T_i)$, where $T_i$ is a measured temperature value and $\hat{T}$ is the neural network output that depends only on the network weights $\mathbf{W}$ and spatial coordinates $(x, y, z)$, but not on $k_{ii}$. Consequently, automatic differentiation has no pathway to compute $\frac{\partial L_{\text{Data}}}{\partial k_{ii}}$ within this formulation. Instead, the backward finite-difference approach provides a pragmatic means to estimate the descent direction by referencing historical changes in both data-loss values and parameter updates across the epochs.

$$k_{ii}^{(i+1)} = k_{ii}^{(i)} - \beta \cdot \delta k_{ii}^{(i)} \tag{13}$$

where $k_{ii}^{(i)}$ and $k_{ii}^{(i-1)}$ are the parameter values at iterations $i$ and $i-1$ respectively, $\beta$ is the learning rate, and $\delta k_{ii}$ is the pseudo gradient of $\mathscr{L}_{Data}$ with respect to $k_{ii}$.

$$\delta k_{ii}^{(i)} = \frac{(\mathscr{L}_{\text{Data}}^{(i)} - \mathscr{L}_{\text{Data}}^{(i-1)})}{(k_{ii}^{(i)} - k_{ii}^{(i-1)}) + \gamma} \tag{14}$$

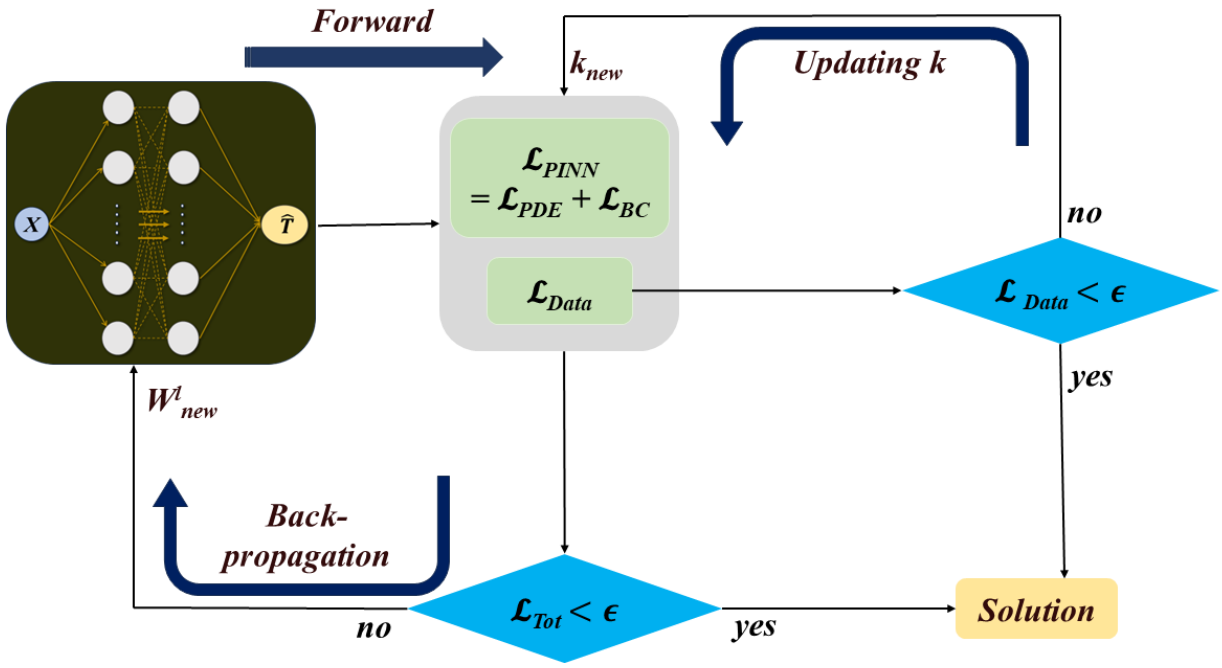



Figure 2: Schematic representation of the modified inverse-physics-informed neural network

Here, $\mathscr{L}_{\text{Data}}^{(i)}$ and $\mathscr{L}_{\text{Data}}^{(i-1)}$ are the data losses at iterations $i$ and $i-1$ respectively. These two formulae, together, enable iterative estimation and updating of the parameter using both gradient descent and finite-difference approximations.

It is to be noted that the denominator $(k_{ii}^{(i)} - k_{ii}^{(i-1)})$ in the Equation (14) may indeed become very small during training, which may cause numerical instability. Therefore, a very small stabilizing constant $\gamma$ is added to the denominator term at every epoch to avoid blowing up of the pseudo gradient. It should be taken care that the stabilizing constant $\gamma$ is sufficiently small to avoid biasing the descent direction while maintaining robustness. In this work, a stabilizing constant of $5 \times 10^{-6}$ was used, which is sufficiently small to avoid biasing the descent direction while maintaining robustness.

The algorithm initiates by specifying the geometry, collocation points, boundary conditions, and data points. Subsequently, the neural network architecture is defined with a required number of neurons and layers. Unknown parameters are initialized based on informed estimations. The total loss function is formulated as the weighted sum of PDE loss, boundary loss, and data loss.

Network weights are updated based on the total loss using automatic differentiation and an appropriate variant of gradient descent. The unknown physics parameters are adjusted utilizing the first-order finite difference method and history-based pseudo-gradient descent, relying on the data loss component as described. The algorithm continues to run until the loss values stabilize within a suitable tolerance. Initially, all loss weights are set to unity and subsequently adjusted based on the observed relative convergence rates of the data-loss and physics-loss terms. It is noted that improper weighting can often lead to stagnation in local minima, whereas appropriately balanced weights facilitate stable convergence. Also, the learning rate for the network parameters must be sufficiently low so that the PINN solution has enough time to stabilize while the PDE parameter converges through the pseudo-gradient loop. This prevents premature overfitting and improves overall robustness. If the losses converge to local minima, the hyperparameters $(\alpha, \beta, \lambda)$ are perturbed, and the learning process is rerun. This process is repeated until satisfactory convergence is reached. The parameters obtained at convergence represent the solution to the inverse problem.

Figure 2 represents the flow chart, and algorithm 1 shows the detailed workflow of MIPINNs.

**Algorithm 1** Procedure for modified inverse physics-informed neural network

1: Define the geometry: collocation points, boundary points, data points
2: Define the neural network architecture
3: Initialize weights $\mathbf{W}^{\eta}$ and parameters $k$
4: Declare the values for the hyper-parameters $\alpha, \beta, \lambda$
5: **for** each training iteration $i = 1, 2, \ldots$ **do**
6: Compute total loss:

$$\mathscr{L} = \lambda_P \cdot \mathscr{L}_{PDE} + \lambda_B \cdot \mathscr{L}_{BC} + \lambda_D \cdot \mathscr{L}_{Data}$$

7: Update the weights using backpropagation:

$$[\mathbf{W}^{\eta}]^i = [\mathbf{W}^{\eta}]^{i-1} - [\delta \mathbf{W}^{\eta}]^i$$

8: Compute the pseudo gradient of data loss with respect to $k$:

$$\delta k_{ii}^{(i)} = \frac{\mathscr{L}_{Data}^{(i)} - \mathscr{L}_{Data}^{(i-1)}}{(k_{ii}^{(i)} - k_{ii}^{(i-1)}) + \gamma}$$

9: Update parameter $k$:

$$k_{ii}^{(i+1)} = k_{ii}^{(i)} - \beta \cdot \delta k_{ii}^{(i)}$$

10: **if** $\mathscr{L}_{Data}^{(i)} < \varepsilon$ **and** $\mathscr{L}_{tot}^{(i)} < \varepsilon$ **then**
11: Conclude that $k$ has reached $k_{true}$ and the model has found the converged solution
12: **break**
13: **else**
14: Perturb the hyper-parameters $\alpha, \beta, \lambda$
15: Continue training from Step 5
16: **end if**
17: **end for**

In comparision with traditional inverse methods, such approaches are limited in their ability to incorporate the physical principles underlying the problem. Therefore, these approaches generally rely on a large volume of data to estimate the inverse parameters, necessitating numerous prior parametric simulations leading expensive computational effort. In contrast, PINNs do not require any parametric simulations, thereby substantially reducing computational costs. PINNs have the advantage of tackling both the forward and inverse problems simultaneously, whereas traditional inverse methods typically address only the inverse problem.

On the other hand, certain inverse approaches, such as the Bayesian method, provide probabilistic results that account for uncertainties in the input data. This capability is absent in the current MIPINN framework, which operates as a deterministic approach. Additionally, advanced search methods like Markov Chain Monte Carlo (MCMC) used in Bayesian frameworks are well established, straightforward and reliable, but their implementation can be tedious and resource intensive.

# 4 Experimental methodology

The modified inverse-PINN (MIPINN) method is employed to determine the isotropic thermal conductivity of a stainless steel (SS304) slab and the orthotropic components of thermal conductivity of a pouch-type Li-ion battery from actual temperature measurements. Figure 3(a) shows the experimental setup developed for measuring the surface temperature of the test-sections, which is essential for the estimation of thermal conductivities of the test-sections. Both the test sections (3(b) and (c)) have dimensions of $225 \times 168 \times 7.3\ mm^3$. The test-section is heated using a nichrome plate-type heater with a cross-sectional area of $30 \times 30\ mm^2$ mounted on the top surface of the test-section. The heater is powered by a TDK-LAMBDA DC power source whose voltage and current can be fine-tuned depending on the heat input requirement. Calibrated K-type thermocouples with an uncertainty of $\pm 0.25\ K$ are positioned on the surface of the test-section, which measures the temperature, and the temperatures were recorded using Agilent data acquisition unit. Temperature data are collected at 17 locations and 19 locations on the SS304 slab and the pouch-type Li-ion battery, respectively, referring to [47]. The bottom surface of both the test-sections was insulated with a cork block, and the rest of the faces are exposed to the ambient conditions

Experiments are conducted on the test-sections separately by supplying heat inputs of 11 and 15 W to the SS304 slab and the pouch-type Li-ion battery, respectively, in a temperature-controlled environment with an ambient temperature of $300 \pm 0.25\ K$. The temperatures measured by the thermocouples positioned on the surface of the test-section are recorded until the steady-state condition is reached. Steady-state condition is declared when the temperature remain within the uncertainty range of the thermocouples for half an hour. The repeatability of the experiments is ensured by repeating the experiments three times for both the test-sections under the respective heat input conditions, and the maximum deviation in the temperature is found to be less than $\pm 0.50\ K$.

Uncertainty analysis is performed to determine the uncertainty associated with the measured quantities – voltage ($V$), current ($I$), and temperature ($T$), and the derived quantity – heat input ($Q$). The principle of propagation of uncertainty is used to determine the uncertainty in the heat input from the uncertainties of current and voltage. Equations (15) and (16) show the mathematical expression to quantify the uncertainty in the heat input ($\sigma_Q$), where ($\sigma_V$) and ($\sigma_I$) represent the uncertainty in voltage and current, respectively. The results of the uncertainty analysis are shown

in the Table 1.

$$Q = V \times I \tag{15}$$

$$\sigma_Q = \pm\sqrt{\left(\frac{\partial Q}{\partial V}\sigma_V\right)^2 + \left(\frac{\partial Q}{\partial I}\sigma_I\right)^2} \tag{16}$$

Table 1: Results of the uncertainty

| **S.no** | **Parameter** | **Uncertainty** | **Unit** |
|---|---|---|---|
| 1 | Voltage, $V$ | $\pm 0.10$ | V |
| 2 | Temperature, $T$ | $\pm 0.25$ | K |
| 3 | Current, $I$ | $\pm 0.01$ | A |
| 4 | Heat input (SS304) | $\pm 0.15$ | W |
| 5 | Heat input (Battery) | $\pm 0.17$ | W |

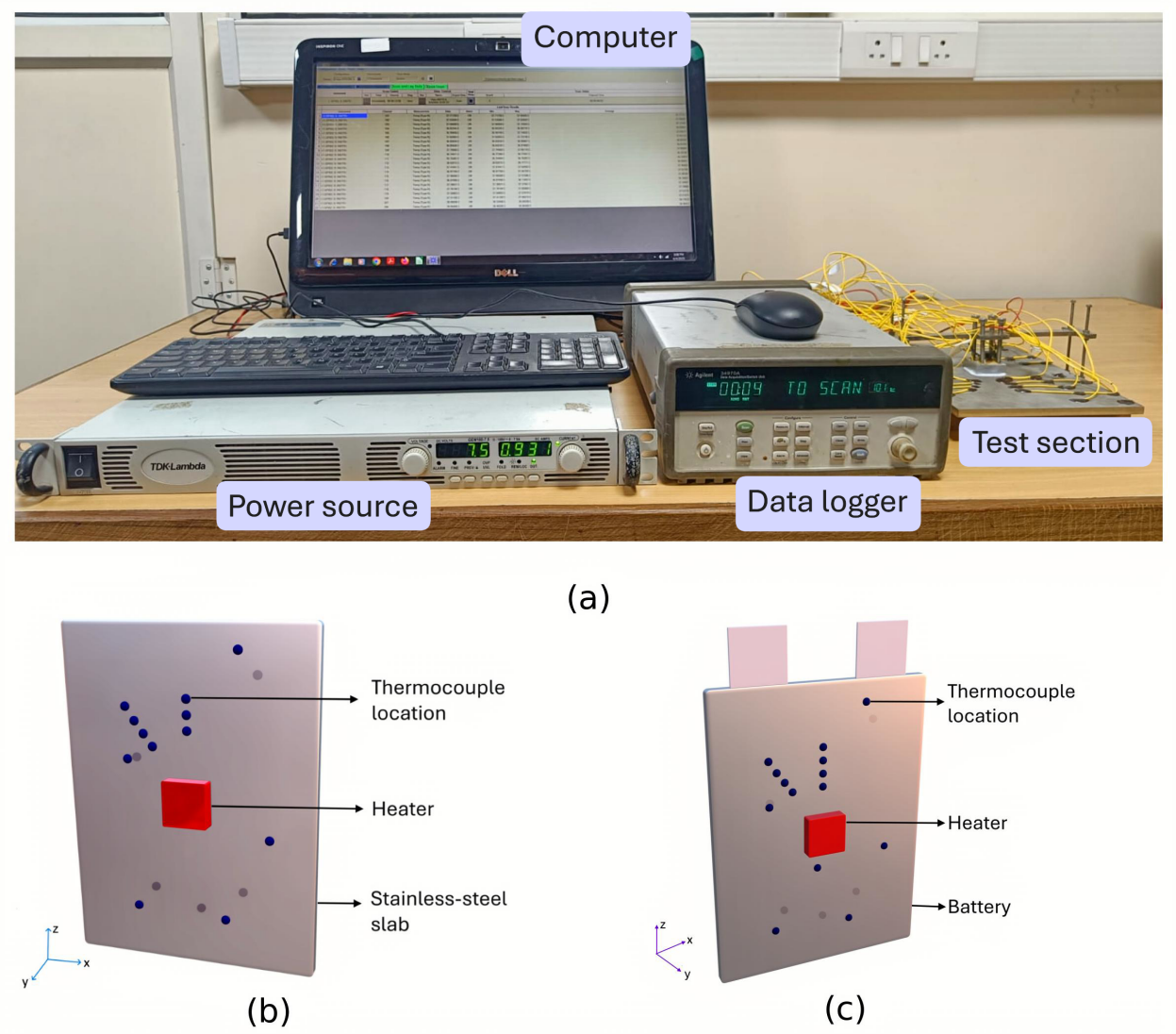



Figure 3: (a) Experimental setup for measuring the temperature at various locations, (b) SS304 test-section depicting the temperature sensor points, (c) Pouch-type Li-ion battery test section depicting the temperature sensor points

# 5 Performance evaluation of MIPINNs in solving inverse heat conduction problems

This study investigated distinct cases to estimate the isotropic and orthotropic components of thermal conductivity of the domain under various boundary conditions, and then presented a case illustrating the scalability of the methodology for solving the transient inverse heat conduction problem to determine the specific heat capacity.

1. Comparison of proposed MIPINN methodology over vanilla inverse PINN methodology in solving inverse-heat conduction by considering temperature to be a known exact solution.

2. Evaluating the robustness of MIPINNs by conducting a three-dimensional numerical inverse-heat conduction experiment.

3. Estimating the thermal conductivity of an SS304 slab through temperature measurements based on in-house experiments.

4. Determining the orthotropic components of thermal conductivity of a pouch-type Li-ion battery.

5. Scalability to three-dimensional transient inverse heat conduction problems.

In all instances, a limited dataset (measured temperatures at a small number of discrete locations) was utilized to determine isotropic and orthotropic components of thermal conductivity. The mean absolute error (MAE) was used as the loss metric for all the following cases. Based on the literature [18, 20, 22] and numerical trials, Adam optimizer and Tanh activation were used for training the NN. All the cases were run on NVIDIA GeForce RTX 4070 GPU.

## Case 1 Estimation of isotropic and orthotropic thermal conductivity by considering temperature as known exact solution

Here, the temperature field was prescribed as an exact analytical solution for the considered cases. This allows for a quantitative assessment of the proposed methodology through direct comparison between the predicted and exact solutions. Furthermore, a systematic comparative analysis was carried out between the vanilla inverse PINN and the MIPINN approaches to evaluate their accuracy, convergence characteristics, and overall computational performance.
Consider the equation (1) which governs the steady-state heat conduction with heat generation $q'''$. In the following cases within this section, the heat generation term $q'''$ is derived from the given by substituting the exact solution $T$ into the equation (1) with the specified orthotropic conductivities. Consequently, thermal conductivities are determined by solving the PDE using the proposed methodology.

### Case 1.1 Estimation of isotropic thermal conductivity of two-dimensional domain having $T = \frac{1}{2}\cos(\pi x)\sin(\pi y)$

A domain of area 1×1 m², as shown in the figure 4, was selected for solving this problem. Here, the top and bottom edges were given Dirichlet conditions, and Neumann condition was given on the remaining edges. A total of 100 collocation points within the domain were utilized to calculate the PDE loss, 50 points on each edge to compute the boundary loss, and 11 points to evaluate the data loss. In this two-dimensional case, the thermal conductivity $k$ was set to 1 $W/mK$. A shallow neural network was employed to address this problem, consisting of 2 input neurons ($x$ and $y$) in the input layer, 21 neurons in the hidden layer, and a single output neuron ($T$) in the output layer. All loss weights were set to 1. Network weights were trained with a learning rate of 0.002, and the PDE parameter was trained with a learning rate of 0.2.

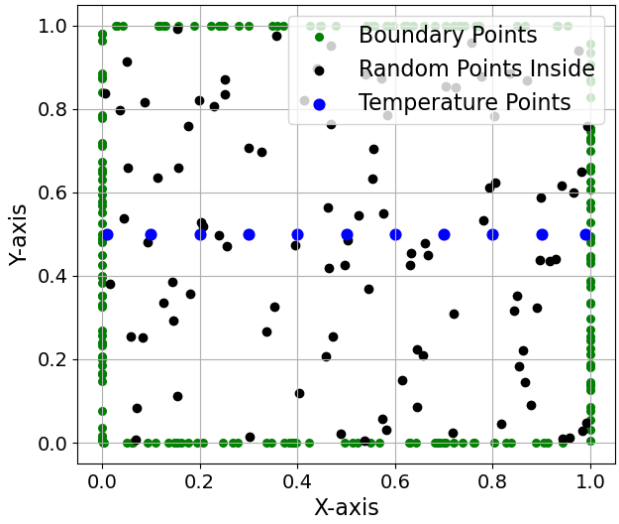



Figure 4: Domain representing collocation points, boundary points and temperature data points for solving the Case 1.1

Figure 5(a) depicts the convergence of all loss values with epochs using the MIPINN methodology, while Figure 5(b) illustrates the convergence of the isotropic thermal conductivity, $k$, determined by MIPINN. The value of $k$ converged within 2000 epochs to $k = 1.034$, which closely approximates the actual value of $k = 1$. In contrast, Figures 5(c) and (d) present the convergence of the losses and $k$ utilizing the standard inverse-PINN approach. It is evident that MIPINNs achieved lower final loss values compared to the conventional inverse-PINN technique. Additionally, MIPINNs required only 9.4 s to achieve full convergence of all losses within 2000 epochs, whereas the vanilla PINN required 14.88 s.

Figure 6 (a) presents the temperature distribution obtained at convergence using the MIPINN methodology. Figure 6 (b) displays the exact solution, while Figure 6 (c) provides the error histogram of the converged solution relative to the exact result. Finally, Figure 6 (d) illustrates the anomaly plot comparing the converged solution with the exact solution for the Case 1.1.

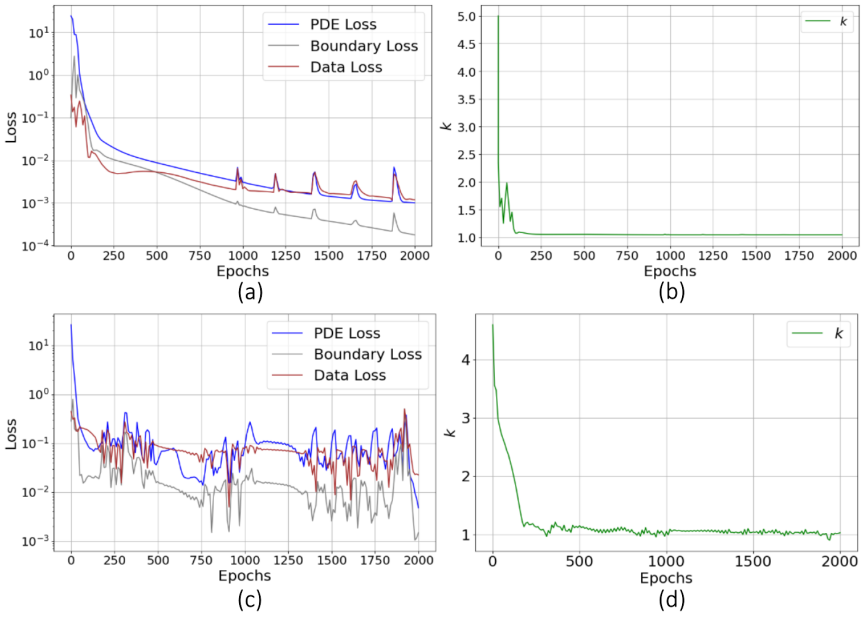



Figure 5: Convergence plot for a) PDE, boundary and data losses with respect to epochs, b) $k$ with respect to epochs using MIPINN, c) PDE, boundary and data losses with respect to epochs and d) $k$ with respect to epochs using vanilla-PINN for the Case 1.1

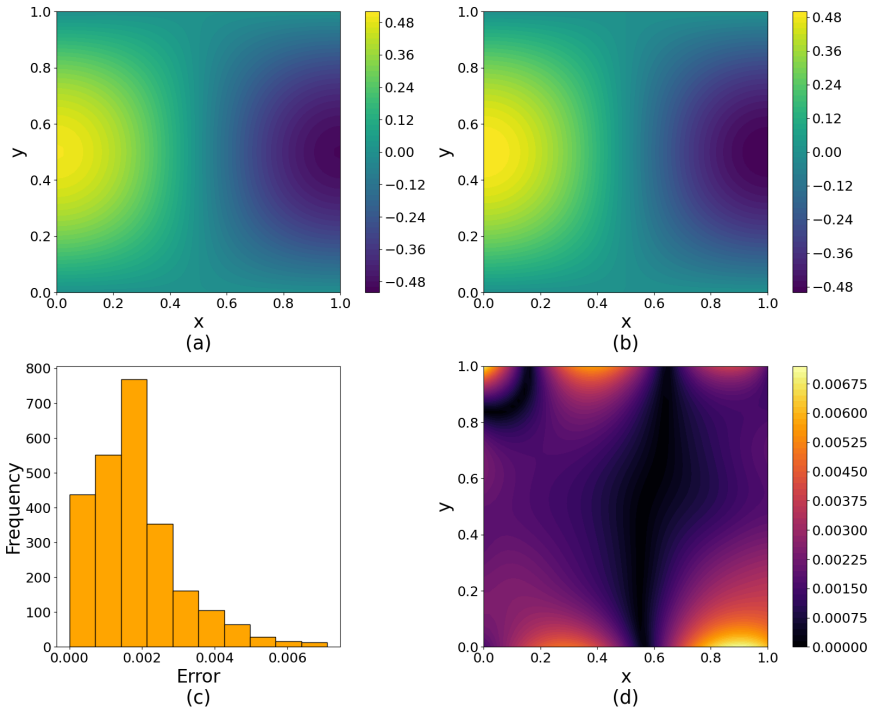



Figure 6: a) Temperature distribution obtained at the convergence using the MIPINN methodology, b) exact solution c) error histogram of the converged solution with respect to the exact solution, d) anomaly plot of the converged solution with respect to solution for the Case 1.1

**Case 1.2 Estimating the orthotropic thermal conductivities of two-dimensional domain having distribution $T = xy \cdot e^{2x+y}$**

This problem focuses on determining the orthotropic thermal conductivities $k_{xx}$ and $k_{yy}$ for $T = xy \cdot e^{2x+y}$ over a 1×1 m² domain. Dirichlet conditions were set on the left and bottom edges, and with Neumann conditions on the others. Similar domain and network configurations to those of the previous case were employed to solve this problem. Here, $k_{xx}$ and $k_{yy}$ were assigned values of 10 $W/mK$ and 3 $W/mK$, respectively. PDE loss weight $\lambda_p$ was set to 0.1 and $\lambda_b, \lambda_d$ were set to 1. Network weights were trained with a learning rate of 0.005, and the PDE parameters were trained with a learning rate of 0.01.

Figure 7(a) shows the convergence of all the losses with respect to epochs using the MIPINN technique. Figure 7(b) shows the convergence of orthotropic conductivity parameters $k_{xx}$ and $k_{yy}$. Convergence was reached within 15000 epochs at $k_{xx} = 9.99$ and $k_{yy} = 2.99$, which are very close to the actual parameters. Figure 7 (c) and (d) shows the convergence of all the losses and the propagation of orthotropic thermal conductivities $k_{xx}$ and $k_{yy}$ with epochs using the vanilla PINN technique.

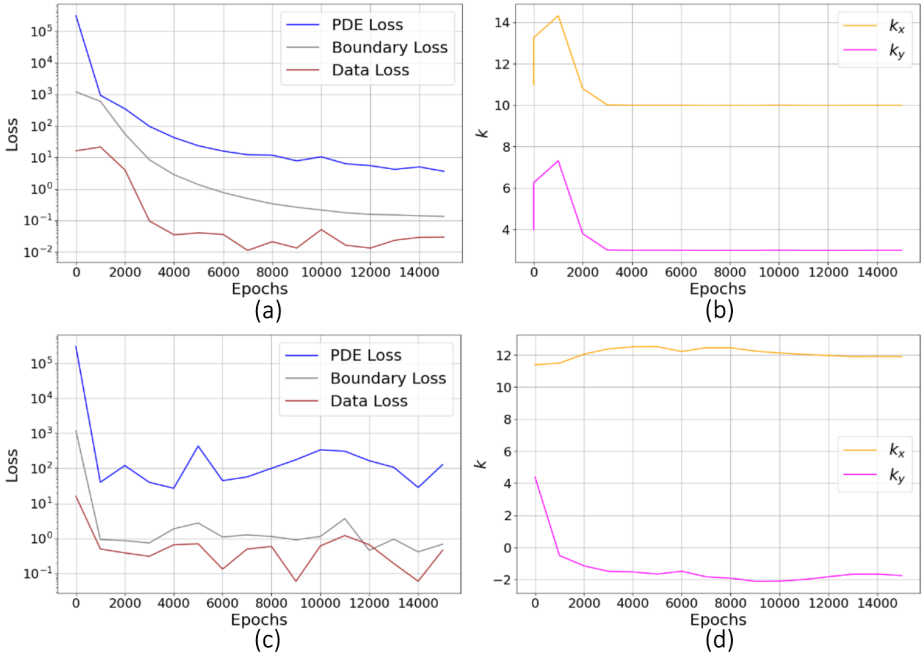



Figure 7: Convergence plot for a) PDE, boundary and data losses with respect to epochs, b) $k$ with respect to epochs using MIPINN, c) PDE, boundary and data losses with respect to epochs and d) $k$ with respect to epochs using vanilla-PINN for the Case 1.2

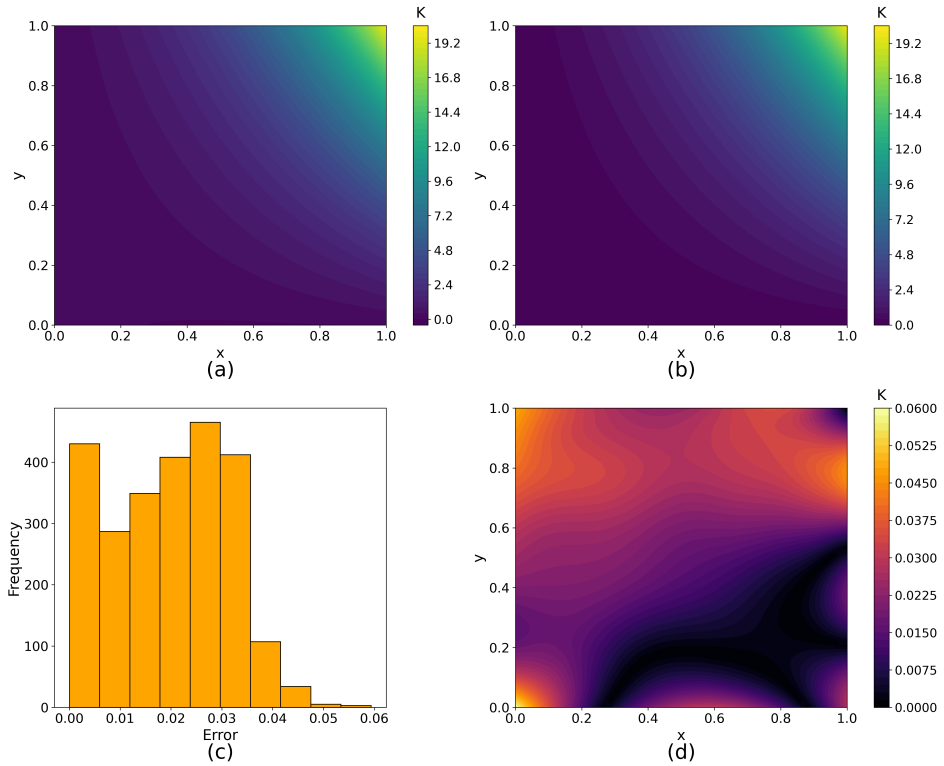



Figure 8: a) Temperature distribution obtained at the convergence using the MIPINN methodology, b) exact solution c) error histogram of the converged solution with respect to the exact solution, d) anomaly plot of the converged solution with respect to solution for the Case 1.2

Figure 8 (a) shows the temperature distribution at convergence using MIPINN. Figure 8 (b) presents the exact solution, 8 (c) gives the error histogram relative to the exact result, and 8 (d) displays the anomaly plot comparing the converged solution with that of the exact solution for Case 1.2.

It is evident that the increase in the number of unknown parameters has led to a higher degree of ill-posedness. Consequently, both the PDE loss and the associated parameters have not achieved convergence, as they are interdependent. In this instance, the vanilla PINN required 92 s to complete 15,000 epochs, whereas the MIPINN required 43.8 s to achieve convergence over 15,000 epochs.

Table 2: Comparison of Vanilla-PINN and MIPINN for Case 1.1 and Case 1.2

| Case | Technique | Epochs | $N_b$ | $M_{coloc}$ | NN | $k_{\text{actual}}$ | $k_{\text{PINN}}$ | time (s) | MAE |
|---|---|---|---|---|---|---|---|---|---|
| Case 1.1 | Vanilla-PINN | 2000 | 100 | 200 | $1 \times 21 \times 1$ | 1 | 1.042 | 14.88 | 0.066 |
| | MIPINN | 2000 | 100 | 200 | $1 \times 21 \times 1$ | 1 | 1.034 | 9.4 | 0.001 |
| Case 1.2 | Vanilla-PINN | 15000 | 100 | 200 | $1 \times 21 \times 1$ | 10, 3 | 11.1, -1.95 | 92 | 0.55 |
| | MIPINN | 15000 | 100 | 200 | $1 \times 21 \times 1$ | 10, 3 | 9.99, 2.99 | 43.8 | 0.054 |

Table 2 compares the performance of the proposed modified inverse-PINN (MIPINN) methodology with the traditional inverse-PINN approach for solving the inverse problem of determining

the isotropic and orthotropic thermal conductivities for two different cases. These results indicate that MIPINNs offer both stronger constraints on the solution and greater computational efficiency than the vanilla PINN method. In both cases, MIPINN was considerably faster than the vanilla PINN approach with higher accuracy as the MIPINNs use a simple finite difference approach for calculating the pseudo gradients $\delta k_{ii}$ while the vanilla PINN uses the standard auto-differentiation algorithm, which is comparatively very sophisticated.

## Case 2 Evaluating the robustness of MIPINNs by conducting a three-dimensional numerical inverse-heat conduction experiments

The effectiveness of the proposed methodology for estimating the orthotropic components of thermal conductivity using the MIPINN approach was also evaluated through three-dimensional numerical heat-conduction experiments. Furthermore, simulations were conducted using the finite element method based COMSOL Multiphysics for comparative analysis.

Heat conduction in a three-dimensional slab measuring $250 \times 200 \times 50$ $mm^3$, as depicted in Figure 9, was analysed using the proposed algorithm. Considering practical scenarios, the in-plane conductivities $k_{xx}$ and $k_{zz}$ were assumed to be identical at 41 $W/mK$, whereas the through-plane thermal conductivity $k_{yy}$ is assumed to be 7 $W/mK$. A heat flux of 2000 $W/m^2$ was applied to the rear face, while all other faces were subjected to ambient conditions with a temperature of 300 $K$ and a heat transfer coefficient of 15 $W/m^2K$. The MIPINN simulation employed 2500 collocation points, 9000 boundary points, and 25 temperature data points were considered on the front face, as illustrated in Figure 9. PDE loss weight $\lambda_p$ was set to 1, $\lambda_b$ was set to 0.1, heat flux boundary loss was set to 1.5, and boundary loss on all the faces was set to 0.1. Network weights were trained with a learning rate 0.005, and the PDE parameter was trained with a learning rate of $5 \times 10^{-6}$.

A neuron independence study was performed to find the optimum number of neurons in a two-layered NN based on the accuracy and computational cost. Multiple MIPINN simulations were run for this case by varying the number of neurons in the NN. Temperature data loss of the domain was compared in each of the cases, and the results are shown in the Table 3. The same strategy was used in determining the NN architecture for the Case 3 and Case 4.

Table 3: Results of the neuron-independence study for Case 2

| S.no | Number of neurons | Data loss (MAE) (K) |
|---|---|---|
| 1 | 15 | 0.0393 |
| 2 | 21 | 0.0281 |
| 3 | 25 | 0.0278 |
| 4 | 30 | 0.0273 |

Keeping both computational accuracy and cost into consideration, a deep neural network with two hidden layers, each comprising 21 neurons, was utilized to estimate the temperature distribution and orthotropic conductivities of the slab. Convergence was achieved within 150000 epochs, and $k_{xx}$ and $k_{yy}$ were found to be 41.01 $W/mK$ and 6.96 $W/mK$ respectively at the convergence, which shows a good concurrence with the actual values.

Figure 10(a) demonstrates the convergence of PDE loss, boundary losses, and data loss, while Figure 10(b) illustrates the convergence of thermal conductivities. The same case was resolved

using COMSOL Multiphysics with a refined mesh comprising 4586 grids, based on grid independence study. The outcomes obtained from MIPINN were then compared with the results of COMSOL Multiphysics simulations. Figure 11(a) depicts the temperature distribution on the xz-plane at y = 25 mm upon convergence of orthogonal thermal conductivities utilizing the proposed methodology, whereas Figure 11(b) presents the solution acquired through COMSOL Multiphysics. Figures 11(c) shows the error histogram of the converged solution with respect to the exact solution for the whole domain, and the Figure11(d) illustrates the anomaly plot of the converged solution relative to the COMSOL solution on the xz-plane at y = 25 mm. From Figure 11, it can be inferred that the performance of MIPINN is quite satisfactory. The inputs and outputs for MIPINNs for solving the Case 2 are presented in the Table 4.

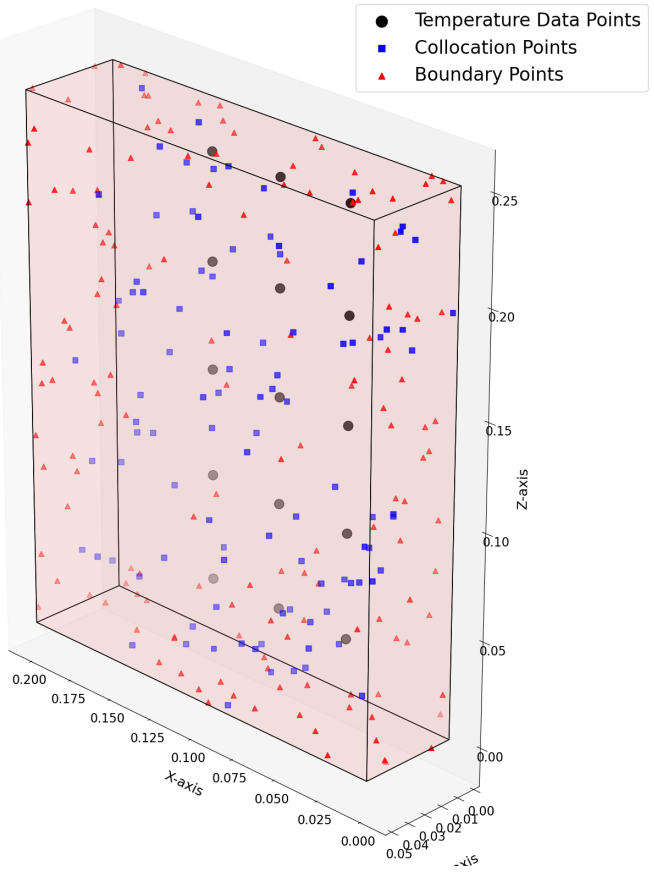



Figure 9: Domain diagram showing the collocation, boundary, and data points for evaluating PDE, boundary, and data loss, respectively, for the Case 2

Table 4: Inputs and outputs for the determination of orthotropic thermal conductivities for the Case 2

| **Case** | **NN** | **$N_{coloc}$** | **$N_b$** | **Epochs** | **$N_{data}$** | **$k_{act}$ (W/mK)** | **$k_{MIPINN}$ (W/mK)** | **Data loss (K)** | **Time (s)** |
|---|---|---|---|---|---|---|---|---|---|
| Case 2 | $2 \times 21 \times 21 \times 1$ | 2500 | 9000 | 150000 | 25 | 41, 7 | 41.01<br>6.96 | 0.028 | 1087 |

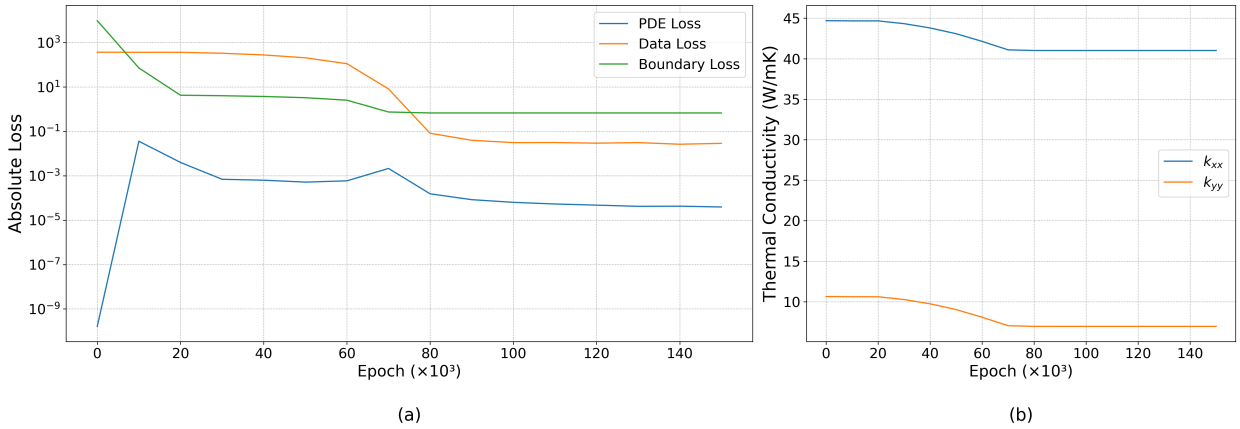



Figure 10: a) Convergence of all the PDE, boundary, and data loss with epochs, b) convergence of the orthotropic components of thermal conductivity for the Case 2

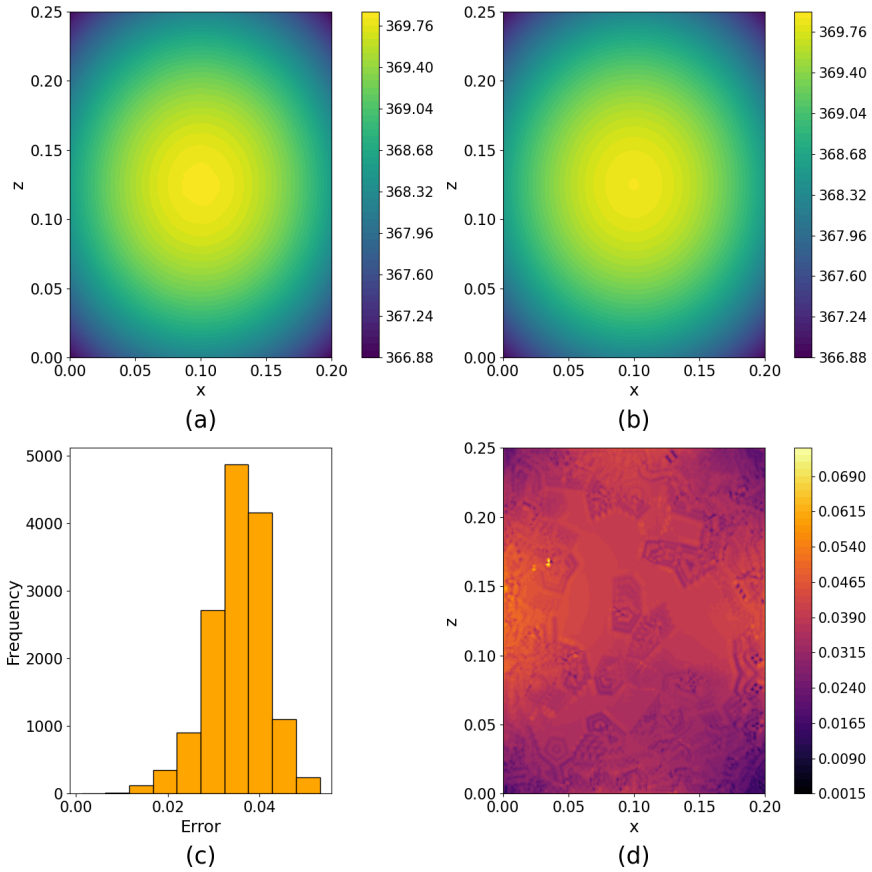



Figure 11: a) Temperature distribution of mid-xz-plane, obtained at the convergence of the orthogonal thermal conductivities using the inverse-PINN methodology, b) solution obtained using COMSOL Multiphysics, c) error histogram of the converged solution with respect to the exact solution, d) anomaly plot of the converged solution with respect to the COMSOL solution for the Case 2

This case was also used to establish the robustness of MIPINNs in solving inverse problems to identify multiple parameters simultaneously by (1) varying the number of temperature data, (2) adding noise to the temperature data, and (3) starting the simulation with a different initialization.

**Robustness Evaluation of MIPINNs in Inverse Problem Solving**

The present case study demonstrates the robustness of MIPINNs for solving inverse problems, particularly for simultaneously identifying multiple parameters. The methodology was evaluated through three key approaches: (1) varying the number of temperature data points, (2) introducing noise into the temperature measurements, and (3) starting simulations with different initial guesses. All the details of the simulations are given in the Table 5.

1. **Impact of Data Quantity** Initial trials involved reducing the number of noise-free temperature data points from 25 to 15, 9, and finally 5. Despite the decrease in data, thermal conductivities consistently converged to the correct values. Furthermore, there was satisfactory convergence observed for the partial differential equation, boundary conditions, and data loss, indicating the method's resilience to reduced data availability.

2. **Effect of Noise in Data** To assess the effectiveness of MIPINNs under noisy conditions, 1% Gaussian noise was added to the temperature data comprising 15 points. The results remained highly satisfactory, showing that the methodology is capable of handling measurement uncertainties without compromising performance.

3. **Variation in Initialization** The robustness of the approach was further tested by initiating the simulations with different guess values for the parameters. Even with varied initializations, the methodology continued to yield very satisfactory results.

Across all trials, the inverse parameters were determined efficiently, thereby substantiating the robustness and reliability of the MIPINNs methodology for solving complex inverse problems.

Table 5: Results for different initial guesses and data sizes for Case 2

| **Initial Guess (W/mK)** | $\mathbf{N_{data}}$ | $\mathbf{k_{xx}}$ **(W/mK)** | $\mathbf{k_{yy}}$ **(W/mK)** | **Data Loss (K)** |
|---|---|---|---|---|
| $k_{xx} = 44.5,\ k_{yy} = 11$ | 25 | 41.011 | 6.96 | 0.028 |
| | 15 | 41.015 | 6.965 | 0.025 |
| | 9 | 41.012 | 6.960 | 0.028 |
| | 5 | 41.000 | 6.950 | 0.008 |
| | 15 (1% noise) | 41.028 | 6.977 | 1.8 |
| $k_{xx} = 77.5,\ k_{yy} = 43.5$ | 15 | 40.96 | 6.964 | 0.020 |

The proposed methodology, after validating with the above numerical heat transfer cases, is now used for determining the isotropic thermal conductivity of a steel slab and orthotropic components of thermal conductivity of a pouch-type battery by conducting actual experiments.

Case 3 describes the estimation of the thermal conductivity of an SS304 slab, and Case 4 illustrates the estimation of the orthotropic components of thermal conductivity of a pouch-type Li-ion battery.

## Case 3 Determination of isotropic thermal conductivity of a stainless steel (SS304) slab

The goal of this study is to determine the isotropic thermal conductivity of an SS304 slab using the MIPINN approach proposed in this work with actual temperature measurements. The slab surface opposite to the heater was insulated, while all other surfaces were exposed to ambient conditions. Experiments were conducted with a heat load of 11 $W$ and were performed three times to ensure repeatability. Temperature measurements were taken at 17 locations on the slab as shown in the Figure 3 (b). Non-dimensionalised forms of the governing equation and boundary conditions were used to solve this problem. The MIPINN model utilised a deep neural network with 2 hidden layers of 150 neurons each, taking (x, y, z) as inputs and outputting $\hat{T}$. For the simulation, the network utilized 8000 boundary points, 3000 collocation points, and 17 temperature data points out of which one was observed as an outlier. PDE loss weight $\lambda_p$ was set to 1, $\lambda_b$ was set to 0.1, heat flux boundary loss was set to 2.5, and boundary loss on all the faces was set to 0.1. Network weights were trained with a learning rate of 0.001, and the PDE parameter was trained with a learning rate of 0.1.

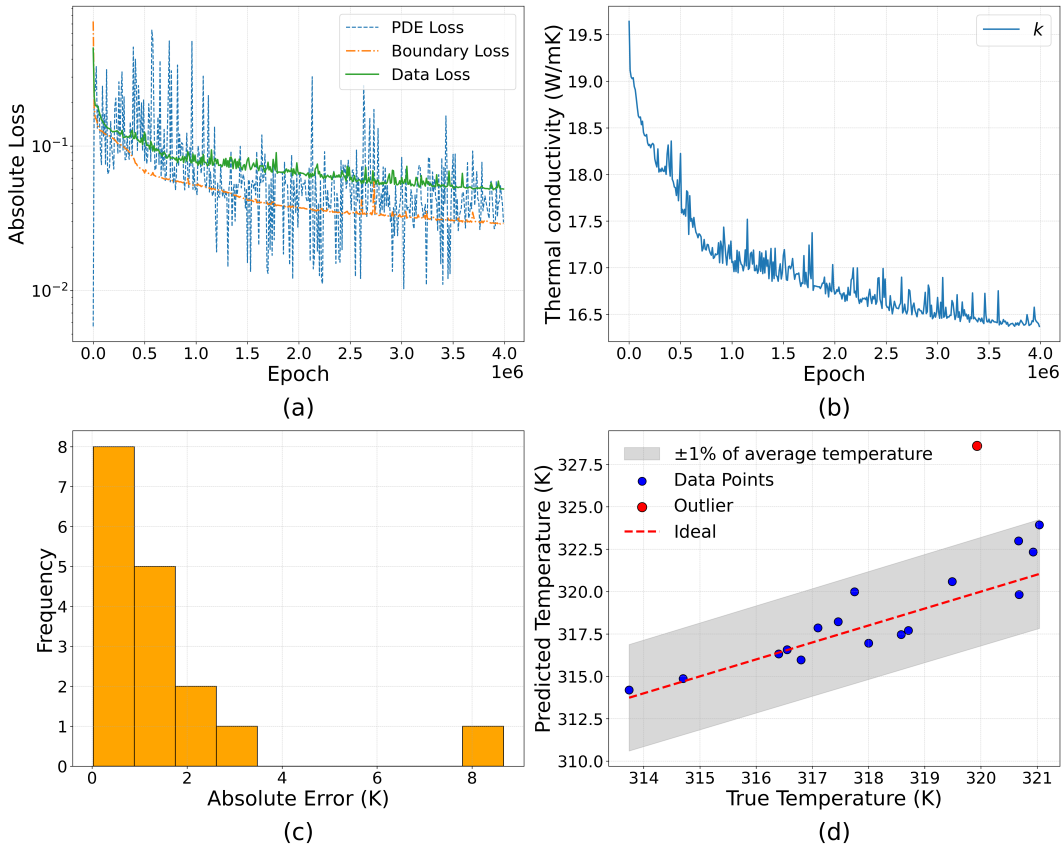



Figure 12: a) Convergence of PDE, boundary and data losses with epochs, b) Convergence of isotropic conductivity of SS304 slab, c) histograms of the error between predicted and measured temperatures, d) parity plot for comparing predicted and measured temperatures for the Case 3

Figure 12(a) illustrates the convergence of all the boundary losses, PDE loss, and the data loss, while Figure 12(b) shows the convergence of thermal conductivity. Figure 12(c) displays the histograms of the absolute error between predicted and measured temperatures, and Figure 12(d) provides parity plots comparing the predicted and measured temperatures. All plots confirm that: (1) the solution has converged, and (2) estimated thermal conductivity values are accurate. All the inputs and results for the MIPINN model for solving the Case 3 are presented in the Table 6. Upon removing the outlier, the data loss converged at 1.06 $K$ with a standard deviation of 1.25 and a coefficient of determination ($R^2$) of 0.77. All predicted temperatures at the 17 measured points are within $\pm 1\%$ of the average measured temperature range. The true thermal conductivity of SS304 is $15.96 \pm 0.74$ $W/mK$ as measured by a modified transient plane source experimental facility, available at IIT Madras [48], and standard handbook values report approximately 16.2 W/mK. while MIPINN predicts a conductivity of $k = 16.37$ $W/mK$. The results indicate that MIPINN predicts values for thermal conductivity very close to the tested value, although the temperature data may be influenced by experimental uncertainties.

## Case 4 Determination of orthotropic thermal conductivity of a Li-ion pouch type Battery

The validated MIPINN methodology was finally employed to also determine the orthotropic components of thermal conductivity of a Li-ion pouch-type battery. Steady-state temperature data were collected at 19 locations encompassing both the heater side surface and the opposite surface of the battery as shown in Figure 3(c). This data was subsequently utilized to calculate data loss through the MIPINN methodology. In this study, the efficacy of MIPINN methodology was examined in estimating the orthotropic components of thermal conductivity of the battery. Even here, non-dimensional forms of equations were used to solve the problem. The MIPINN model utilized a deep neural network with 2 hidden layers of 150 neurons each, taking (x, y, z) as inputs and outputting $\hat{T}$. A total of 3000 colocation points, 8000 boundary points, and 19 data points were used in solving this case. PDE loss weight $\lambda_p$ was set to 100, $\lambda_b$ was set to 0.1, heat flux boundary loss was set to 2.5, and boundary loss on all the faces was set to 0.1. Network weights were trained with a learning rate of 0.0008, and the PDE parameter was trained with a learning rate of 0.0005.

Figure 13(a) shows the convergence of all the boundary, PDE, and data losses. Figure 13(b) presents the convergence of thermal conductivities. Figure 13(c) displays histograms of the absolute error between predicted and measured temperatures. Additionally, Figure 13(d) provides parity plots comparing predicted and measured temperatures. All predicted temperatures at the 19 measured points except one point fall within $\pm 1\%$ of the average measured temperature range. The reported thermal conductivities of the battery are $k_{xx} = k_{zz} = 20.8 \pm 1.4\ W/mK$ and $k_{yy} = 2.9 \pm 0.2\ W/mK$ using Bayesian and Metropolis Hastings–Markov Chain Monte Carlo (MHM-CMC) approach with the same data used here while literature reports in-plane and through-plane conductivities of 26.6 W/mK and 0.52 W/mK, respectively [47]. While MIPINN predicts conductivities of $k_{xx} = k_{zz} = 21.46\ W/mK$ and $k_{yy} = 2.46\ W/mK$ for a heat load of 15 $W$. MIPINN achieves convergence of data loss and good accuracy for thermal conductivity, within the limitations of experimental uncertainties affecting the temperature data. After excluding the outlier, the data-loss metric converged to 1.11K, accompanied by a standard deviation of 1.36 and $R^2$ = 0.89. The inputs and results for the MIPINN model for solving the Case 4 are presented in the Table 6.

Table 6: Inputs and outputs for the determination of thermal conductivities of Case 3 and Case 4

| Case | NN | $N_{coloc}$ | $N_b$ | Epochs | $N_{data}$ | $k_{act}$ (W/mK) | $k_{MIPINN}$ (W/mK) | Data loss (K) | STD | $R^2$ | Time (mins) |
|---|---|---|---|---|---|---|---|---|---|---|---|
| **Case 3** | $3 \times 150 \times 150 \times 1$ | 3000 | 8000 | $40 \times 10^5$ | 17 | $15.96 \pm 0.74$ | 16.37 | 1.06 | 1.25 | 0.77 | 718 |
| **Case 4** | $3 \times 150 \times 150 \times 1$ | 3000 | 8000 | $15 \times 10^5$ | 19 | $20.8 \pm 1.44$<br>$2.9 \pm 0.2$ | 21.46, 2.46 | 1.11 | 1.36 | 0.89 | 480 |

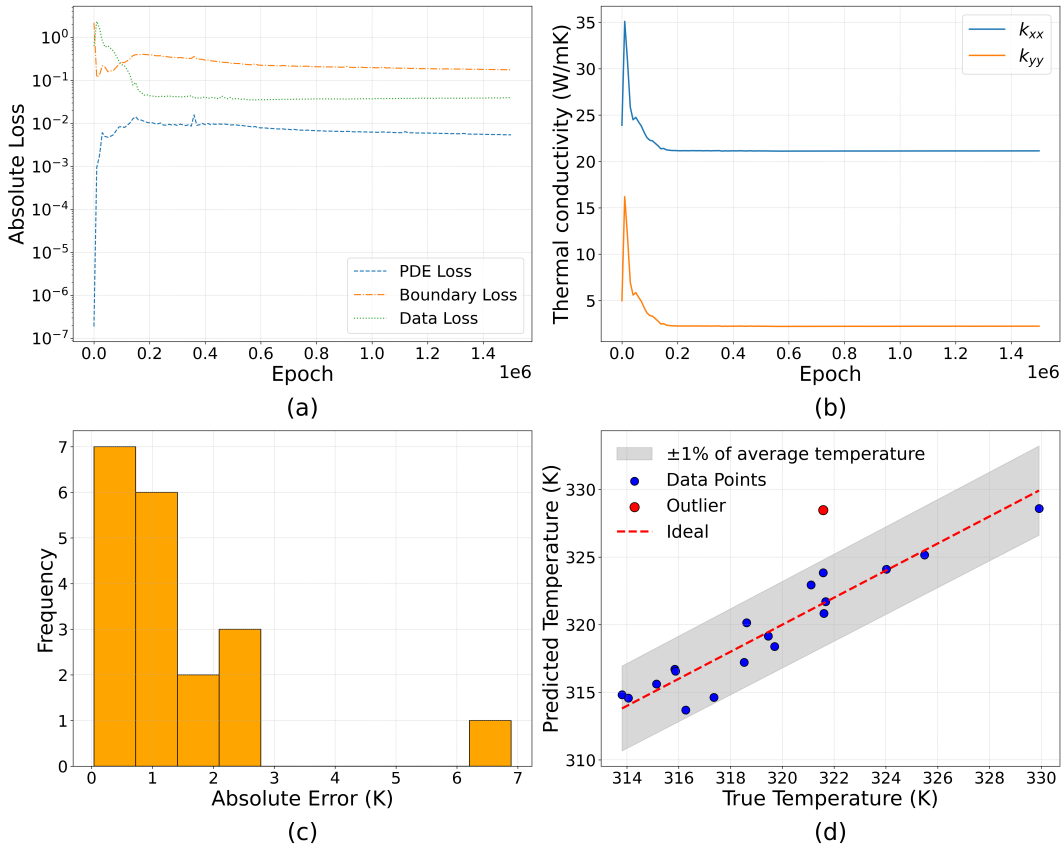



Figure 13: a) Convergence of PDE, boundary, and data losses with epochs, b) Convergence of orthotropic conductivities of battery, c) histograms of the error between predicted and measured temperatures, d) parity plot for comparing predicted and measured temperatures for the Case 4.

## Case 5 Scalability to three-dimensional transient inverse heat diffusion problems

The transient orthotropic heat conduction process is governed by the equation (17). Here, $c_p$ and $\rho$ are the specific heat capacity and density of the domain, respectively. $T$ represents the temperature, a function of the spatial coordinates $X = (x,\ y,\ z)$ and time $t$.

$$\rho c_p \frac{\partial T(X,t)}{\partial t} = k_{xx}\frac{\partial^2 T(X,t)}{\partial x^2} + k_{yy}\frac{\partial^2 T(X,t)}{\partial y^2} + k_{zz}\frac{\partial^2 T(X,t)}{\partial z^2} \tag{17}$$

MIPINN was employed to estimate the domain's specific heat capacity $c_p$ in Case 2. The true value was set to 200 J/(kg·K), and the same geometric and boundary configuration as in Case 2 were used. A fully connected network with architecture $(4 \times 21 \times 21 \times 21 \times 21 \times 1)$ and SiLU activation functions was trained using $(x, y, z, t)$ as inputs and $\hat{T}$ as the output. Temperature measurements at 24 locations, contaminated with 1% Gaussian noise, were used for the inverse estimation.

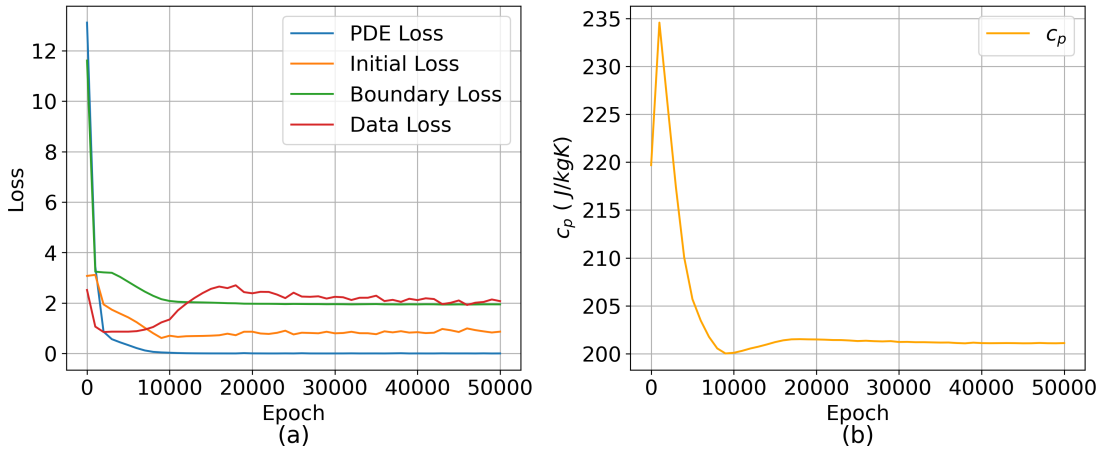



Figure 14: (a) Convergence of physics losses and the data loss for the Case 5, (b) Convergence of $c_p$ with epochs

The parameter $c_p$ converged to 201.1 J/(kg·K) within 50,000 epochs, requiring approximately 150 minutes of computation. Figure 14(a) shows the evolution of the physics and data losses, while Figure 14(b) illustrates the convergence of $c_p$ for Case 5. Future work may incorporate advanced forward PINN architectures to further accelerate the computational process.

# 6 Conclusions

This work presents a modified inverse physics-informed neural network (MIPINN) framework for the identification of the principal components of the thermal conductivity tensor in engineering materials. The framework is designed to improve the robustness of parameter identification in multi-parameter inverse problems.

The MIPINN framework is validated through four progressively complex scenarios: an exact temperature distribution case for rigorous performance evaluation, a synthetic three-dimensional orthotropic heat conduction problem, experimental estimation of isotropic thermal conductivity in AISI 304 stainless steel, and orthotropic thermal property characterization of AMP20M1HD-A lithium-ion battery pouch cells.

Key contributions and findings of this work include:

**Methodological Improvements:** The proposed MIPINN framework employs a two-loop optimization strategy that improves constraint enforcement, enhances solution stability, and reduces computational overhead compared to conventional inverse-PINNs. It was observed that vanilla inverse PINNs successfully identified the parameter in the isotropic diffusion case but failed to capture orthotropic parameters due to insufficient constraints, thereby demonstrating the superiority of MIPINNs for determining orthotropic inverse parameters.

**Validation Results:** We show that MIPINN consistently performs well across increasingly practical cases. Firstly, synthetic case validation confirmed excellent agreement between predicted and ground-truth thermal conductivity values with rapid convergence characteristics. Further, experimental validation on SS304 specimens yielded thermal conductivity estimates of 16.37 W/mK,

demonstrating notable accuracy against reference measurements ($15.96 \pm 0.74$ W/mK, relative error $< 2.6\%$).

The identification of orthotropic thermal conductivity for the lithium-ion battery cells yielded estimates of $k_{xx} = 21.46$ W$\cdot$m$^{-1}$K$^{-1}$ and $k_{yy} = 2.46$ W$\cdot$m$^{-1}$K$^{-1}$. These results demonstrate a consistent estimation of in-plane conductivity when compared to the reported reference values of $k_{xx} = k_{zz} = 20.8 \pm 1.4$ W$\cdot$m$^{-1}$K$^{-1}$ and $k_{yy} = 2.9 \pm 0.2$ W$\cdot$m$^{-1}$K$^{-1}$. While a moderate deviation is observed in the through-plane direction, the estimates successfully characterize the expected anisotropic behavior inherent to layered battery architectures.

**Robustness and Performance:** The framework demonstrates consistent robustness to experimental measurement uncertainties and maintains reasonable accuracy under noisy data conditions. Computational efficiency analysis reveals significant performance improvements over the standard inverse PINN method, with accelerated convergence and reduced memory requirements.

# 7 Outlook

The methodology can be extended to in-house transient thermal parameter identification, multiphysics inverse problems, and real-time thermal property monitoring in operational systems. The framework's architecture is inherently adaptable to other governing equations beyond heat conduction, suggesting broad applicability in the domain of thermal sciences.

Future research directions include implementation of adaptive loss weighting schemes, extension to time-dependent parameter estimation in heat transfer, and integration with uncertainty quantification methodologies for enhanced reliability in engineering design applications.

**Acknowledgment:**
The authors thank Prof. K.S. Reddy, IIT Madras for facilitating the modified transient plane source experimental setup for measuring the thermal conductivity.

# References

[1] J. V. Beck, B. Blackwell, and C. R. S. Clair, *Inverse heat conduction: Ill-posed problems*. James Beck, 1985.

[2] B. Sawaf and M. Özisik, "Determining the constant thermal conductivities of orthotropic materials by inverse analysis," *International Communications in Heat and Mass Transfer*, vol. 22, no. 2, pp. 201–211, 1995.

[3] M. Monde, "Analytical method in inverse heat transfer problem using laplace transform technique," *International Journal of Heat and Mass Transfer*, vol. 43, no. 21, pp. 3965–3975, 2000.

[4] C.-S. Liu, "Identification of temperature-dependent thermophysical properties in a partial differential equation subject to extra final measurement data," *Numerical Methods for Partial Differential Equations: An International Journal*, vol. 23, no. 5, pp. 1083–1109, 2007.

[5] M. Mierzwiczak and J. Kołodziej, "The determination temperature-dependent thermal conductivity as inverse steady heat conduction problem," *International Journal of Heat and Mass Transfer*, vol. 54, no. 4, pp. 790–796, 2011.

[6] B. Czel and G. Grof, "Inverse identification of temperature-dependent thermal conductivity via genetic algorithm with cost function-based rearrangement of genes," *International Journal of Heat and Mass Transfer*, vol. 55, no. 15-16, pp. 4254–4263, 2012.

[7] F.-B. Liu, "A modified genetic algorithm for solving the inverse heat transfer problem of estimating plan heat source," *International Journal of Heat and Mass Transfer*, vol. 51, no. 15-16, pp. 3745–3752, 2008.

[8] M. Hematiyan, A. Khosravifard, and Y. Shiah, "A novel inverse method for identification of 3d thermal conductivity coefficients of anisotropic media by the boundary element analysis," *International Journal of Heat and Mass Transfer*, vol. 89, pp. 685–693, 2015.

[9] J. Wang and N. Zabaras, "Hierarchical bayesian models for inverse problems in heat conduction," *Inverse Problems*, vol. 21, no. 1, p. 183, 2004.

[10] F. Zhu, J. Chen, Y. Han, and D. Ren, "A deep learning method for estimating thermal boundary condition parameters in transient inverse heat transfer problem," *International Journal of Heat and Mass Transfer*, vol. 194, p. 123089, 2022.

[11] S. Kumar, R. Akula, and C. Balaji, "An inverse methodology to estimate the thermal properties and heat generation of a li-ion battery," *Applied Thermal Engineering*, vol. 236, p. 121752, 2024.

[12] M. Hafid, M. Lacroix, and N. Hebbir, "Inverse heat transfer prediction of the thermal parameters of tumors during cryosurgery," *International Journal of Thermal Sciences*, vol. 192, p. 108453, 2023.

[13] S. Chanda, C. Balaji, S. Venkateshan, A. Ambirajan, and V. Ramakrishnan, "Simultaneous estimation of principal thermal conductivities of an anisotropic composite medium: an inverse analysis," *Journal of Heat Transfer*, vol. 135, no. 2, p. 021301, 2013.

[14] C.-H. Huang and Y. Jan-Yuan, "An inverse problem in simultaneously measuring temperature-dependent thermal conductivity and heat capacity," *International Journal of Heat and Mass Transfer*, vol. 38, no. 18, pp. 3433–3441, 1995.

[15] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational Physics*, vol. 378, pp. 686–707, 2019.

[16] V. Dwivedi, N. Parashar, and B. Srinivasan, "Distributed learning machines for solving forward and inverse problems in partial differential equations," *Neurocomputing*, vol. 420, pp. 299–316, 2021.

[17] A. D. Jagtap and G. E. Karniadakis, "Extended physics-informed neural networks (xpinns): A generalized space-time domain decomposition based deep learning framework for nonlinear partial differential equations," *Communications in Computational Physics*, vol. 28, no. 5, 2020.

[18] S. Manikkan and B. Srinivasan, "Transfer physics informed neural network: a new framework for distributed physics informed neural networks via parameter sharing," *Engineering with Computers*, vol. 39, no. 4, pp. 2961–2988, 2023.

[19] A. G. Neelan, G. S. Krishna, and V. Paramanantham, "Physics-informed neural networks and higher-order high-resolution methods for resolving discontinuities and shocks: A comprehensive study," *Journal of Computational Science*, vol. 83, p. 102466, 2024.

[20] A. D. Jagtap, Z. Mao, N. Adams, and G. E. Karniadakis, "Physics-informed neural networks for inverse problems in supersonic flows," *Journal of Computational Physics*, vol. 466, p. 111402, 2022.

[21] S. Cai, Z. Mao, Z. Wang, M. Yin, and G. E. Karniadakis, "Physics-informed neural networks (pinns) for fluid mechanics: A review," *Acta Mechanica Sinica*, vol. 37, no. 12, pp. 1727–1738, 2021.

[22] S. Cai, Z. Wang, S. Wang, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks for heat transfer problems," *Journal of Heat Transfer*, vol. 143, no. 6, p. 060801, 2021.

[23] X. Zhang, C. Tu, and Y. Yan, "Physics-informed neural network simulation of conjugate heat transfer in manifold microchannel heat sinks for high-power igbt cooling," *International Communications in Heat and Mass Transfer*, vol. 159, p. 108036, 2024.

[24] T. Wang, G. Liu, E. Li, and X. Xu, "Adaptive deep physics-informed neural network with dual-nested activation for solving complex partial differential equations," *Computer Methods in Applied Mechanics and Engineering*, vol. 444, p. 118125, 2025.

[25] J. Xu, H. Wei, and H. Bao, "Physics-informed neural networks for studying heat transfer in porous media," *International Journal of Heat and Mass Transfer*, vol. 217, p. 124671, 2023.

[26] B.-E. Madir, F. Luddens, C. Lothodé, and I. Danaila, "Physics informed neural networks for heat conduction with phase change," *International Journal of Heat and Mass Transfer*, vol. 252, p. 127430, 2025.

[27] T. Anandh, D. Ghose, H. Jain, P. Sunkad, S. Ganesan, and V. John, "Improving hp-variational physics-informed neural networks for steady-state convection-dominated problems," *Computer Methods in Applied Mechanics and Engineering*, vol. 438, p. 117797, 2025.

[28] N. Jha and E. Mallik, "Fisher's information matrix approach for fourier features physics-informed neural networks for two-dimensional local time-fractional anomalous diffusion equations with nonlinear thermal diffusivity," *International Journal of Numerical Methods for Heat & Fluid Flow*, vol. 35, no. 8, pp. 2841–2863, 2025.

[29] G.-Z. Zeng, Z.-W. Chen, Y.-Q. Ni, and E.-Z. Rui, "Investigating embedded data distribution strategy on reconstruction accuracy of flow field around the crosswind-affected train based on physics-informed neural networks," *International Journal of Numerical Methods for Heat & Fluid Flow*, vol. 34, no. 8, pp. 2963–2985, 2024.

[30] E.-Z. Rui, G.-Z. Zeng, Y.-Q. Ni, Z.-W. Chen, and S. Hao, "Time-averaged flow field reconstruction based on a multifidelity model using physics-informed neural network (pinn) and nonlinear information fusion," *International Journal of Numerical Methods for Heat & Fluid Flow*, vol. 34, no. 1, pp. 131–149, 2024.

[31] J. Wang, Y. Mo, B. Izzuddin, and C.-W. Kim, "Exact dirichlet boundary physics-informed neural network epinn for solid mechanics," *Computer Methods in Applied Mechanics and Engineering*, vol. 414, p. 116184, 2023.

[32] D. Dalton, D. Husmeier, and H. Gao, "Physics-informed graph neural network emulation of soft-tissue mechanics," *Computer Methods in Applied Mechanics and Engineering*, vol. 417, p. 116351, 2023.

[33] N. Mohammadi, M. A. Zaky, M. Abbaszadeh, and M. Dehghan, "Gradient-enhanced physics-informed neural networks for numerical approximation and parameter estimation of reaction–diffusion systems with chemotaxis and nonlocal terms," *International Journal of Numerical Methods for Heat & Fluid Flow*, vol. 35, no. 11, pp. 3924–3951, 2025.

[34] M. Ghalambaz, M. A. Sheremet, M. A. Khan, Z. Raizah, and J. Shafi, "Physics-informed neural networks (pinns): application categories, trends and impact," *International Journal of Numerical Methods for Heat & Fluid Flow*, vol. 34, no. 8, pp. 3131–3165, 2024.

[35] X. Li, W. Zhao, Q. Li, and Y. Shen, "L-fpwnn: a convlstm-pinn hybrid network with adaptive loss weighting for space-time fractional tsunami modeling," *International Journal of Numerical Methods for Heat & Fluid Flow*, vol. 35, no. 11, pp. 4174–4198, 2025.

[36] V. Oommen and B. Srinivasan, "Solving inverse heat transfer problems without surrogate models: a fast, data-sparse, physics informed neural network approach," *Journal of Computing and Information Science in Engineering*, vol. 22, no. 4, p. 041012, 2022.

[37] M. M. Billah, A. I. Khan, J. Liu, and P. Dutta, "Physics-informed deep neural network for inverse heat transfer problems in materials," *Materials Today Communications*, vol. 35, p. 106336, 2023.

[38] Z. He, F. Ni, W. Wang, and J. Zhang, "A physics-informed deep learning method for solving direct and inverse heat conduction problems of materials," *Materials Today Communications*, vol. 28, p. 102719, 2021.

[39] L. Papadopoulos, S. Bakalakos, S. Nikolopoulos, I. Kalogeris, and V. Papadopoulos, "A computational framework for the indirect estimation of interface thermal resistance of composite materials using xpinns," *International Journal of Heat and Mass Transfer*, vol. 200, p. 123420, 2023.

[40] J. Jo, Y. Jeong, J. Kim, and J. Yoo, "Thermal conductivity estimation using physics-informed neural networks with limited data," *Engineering Applications of Artificial Intelligence*, vol. 137, p. 109079, 2024.

[41] S. Liu, W. Li, X. Hao, X. Hou, and X. Feng, "An inverse prediction and experimental study for the internal insulator boundary conditions of solid rocket motor," *International Journal of Thermal Sciences*, vol. 218, p. 110096, 2025.

[42] M. Berardi, F. V. Difonzo, and M. Icardi, "Inverse physics-informed neural networks for transport models in porous materials," *Computer Methods in Applied Mechanics and Engineering*, vol. 435, p. 117628, 2025.

[43] K. Hornik, M. Stinchcombe, and H. White, "Multilayer feedforward networks are universal approximators," *Neural Networks*, vol. 2, no. 5, pp. 359–366, 1989.

[44] L. Lu, X. Meng, Z. Mao, and G. E. Karniadakis, "Deepxde: A deep learning library for solving differential equations," *SIAM review*, vol. 63, no. 1, pp. 208–228, 2021.

[45] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard *et al.*, "Tensorflow: A system for large-scale machine learning," in *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, 2016, pp. 265–283.

[46] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, 2019, pp. 8024–8035.

[47] J. Jithu, K. N. Vasista, S. Kumar, B. Srinivasan, and C. Balaji, "An inverse methodology to estimate the orthotropic thermal conductivities of amp20m1hd-a pouch-type li-ion battery," *International Journal of Heat and Mass Transfer*, vol. 242, p. 126804, 2025.

[48] S. Jayachandran and K.S.Reddy, "Estimation of out-of-plane effective thermal conductivity of wire mesh using 3d unit-cell model incorporating secondary effects," *Journal of Thermal Science and Engineering Applications*, vol. 15, no. 4, p. 041008, 2023.